

ПРАКТИЧЕСКАЯ РАБОТА №5. СОЗДАНИЕ ЗАПРОСОВ НА ВЫБОРКУ. ОТБОР СТРОК ПО УСЛОВИЮ

5.1. Цель практической работы

Изучить используемый в реляционных СУБД оператор извлечения данных из таблиц. Получить навыки работы с оператором SELECT в программе 'SQL Server Managment Studio'.

5.2. Исходные данные

Исходными данными является индивидуальное задание и результат предыдущих практических работ.

5.3. Используемые программы

Программы 'SQL Server Managment Studio'.

5.4. Теоретические сведения

В SQL имеется единственный оператор, который предназначен для выборки данных из базы данных. Оператор относится к подмножеству **DML**.

Ниже приведен почти полный синтаксис оператора **SELECT**.

```
SELECT [DISTINCT | ALL]  
{* | <величина> [, <величина> ...]}  
[INTO :Переменная [, :Переменная ...]]  
FROM <tableref> [, <tableref> ...]  
[WHERE <условие поиска>]  
[GROUP BY Колонка [, Колонка ...]]  
[HAVING <условие поиска>]  
[UNION [ALL] <select_expr>]  
[ORDER BY <список сортировки>;
```

```
<величина> = {Колонка | :Переменная | <константа>  
| <выражение> | <функция>  
| udf ([<величина> [, <величина> ...]])  
| NULL | USER} [AS Псевдоним]
```

```
<константа> = Число | 'Строка'
```

```
<выражение> = SQL выражение, возвращающее единичное значение
```

```
<функция> =  
COUNT (* | [ALL] <величина> | DISTINCT <величина>)  
| SUM ([ALL] <величина> | DISTINCT <величина>)  
| AVG ([ALL] <величина> | DISTINCT <величина>)  
| MAX ([ALL] <величина> | DISTINCT <величина>)  
| MIN ([ALL] <величина> | DISTINCT <величина>)  
| CAST(<величина> AS <тип данных>)  
| UPPER (<величина>)  
| GEN_ID (Имя_Генератора, <величина>)
```

```
<tableref> = {<joined_table> | table | view  
| procedure([<величина> [, <величина> ...]])}
```

[Псевдоним]

<joined_table> = <tableref> <join_type> **JOIN** <tableref>
ON <условие поиска> | (<joined_table>)

<join_type> = [**INNER**] | {**LEFT** | **RIGHT** | **FULL** } [**OUTER**]

<условие поиска> =
<величина> <оператор сравнения>
{<величина> | (<select_one>)}
| <величина> [**NOT**] **BETWEEN** <величина> **AND** <величина>
| <величина> [**NOT**] **LIKE** <величина>
| <величина> [**NOT**] **IN**
(<величина> [, <величина> ...] | <select_list>)
| <величина> **IS** [**NOT**] **NULL**
| <величина> {>= | <=} <величина>
| <величина> [**NOT**] {= | < | >} <величина>
| {**ALL** | **SOME** | **ANY**} (<select_list>)
| **EXISTS** (<select_expr>)
| **SINGULAR** (<select_expr>)
| <величина> [**NOT**] **CONTAINING** <величина>
| <величина> [**NOT**] **STARTING** [**WITH**] <величина>
| (<условие поиска>)
| **NOT** <условие поиска>
| <условие поиска> **OR** <условие поиска>
| <условие поиска> **AND** <условие поиска>

<оператор сравнения> =
{= | < | > | <= | >= | != | < | > | <= | >= | != | < | > | <= | >= | !=}

<select_one> = оператор **SELECT**, выбирающий одну колонку и возвращающий ровно одно значение

<select_list> = оператор **SELECT**, выбирающий одну колонку, возвращающий ноль или много значений

<select_expr> = оператор **SELECT**, выбирающий несколько величин и возвращающий ноль или много значений

<список сортировки> =
{Колонка | Номер}
[**ASC** | **DESC**]
[, <список сортировки> ...]

Некоторые параметры, входящие в этот оператор, описаны в табл. 5.1.

Описание параметров оператора SELECT

Параметр	Описание
DISTINCT ALL	DISTINCT – предотвращает дублирование данных, которые будут извлечены. ALL (по умолчанию) – приведет к извлечению всех
{* <величина> [, <величина> ...]}	Звездочка (*) означает, что надо извлекать все колонки из указанных таблиц. <величина> [, <величина> ...] – извлекает список указанных
INTO :Переменная [, :Переменная ...]	Используется только в триггерах и хранимых процедурах для операторов SELECT, возвращающих не более одной строки. Указывается список переменных, в которые извлекаются
FROM <tableref> [, <tableref> ...]	Указывает список таблиц, просмотров и хранимых процедур, из которых извлекаются данные. Список может включать соединения и соединения могут быть
table	Имя существующей в базе данных таблицы
view	Имя существующего в базе данных просмотра
procedure	Имя существующей хранимой процедуры, предназначенной для использования в операторе SELECT
Псевдоним	Короткое альтернативное имя для таблицы, просмотра или колонки. После описания в <tableref>, псевдоним может использоваться для ссылок на таблицу или просмотр
join_type	Задаёт тип соединения, которое может быть внутренним или внешним
WHERE <условие поиска>	Указывает условие, которое ограничивает количество извлекаемых строк
GROUP BY Колонка [, Колонка ...]	Разбивает результат запроса на группы, содержащие все строки с идентичными значениями указанными в списке
HAVING <условие поиска>	Используется совместно с GROUP BY. Задаёт условие, которое ограничивает количество возвращаемых групп
UNION [ALL]	Объединяет результаты нескольких запросов. Все запросы должны извлекать одинаковое количество столбцов, тип данных каждого столба первого запроса должен совпадать с типом данных других запросов, имена столбцов в разных запросах могут отличаться. Необязательный параметр ALL
ORDER BY <список сортировки>	Указывает колонки, по которым будет производиться сортировка извлекаемых строк. Можно указывать либо имена колонок, либо их порядковые номера в списке извлекаемых колонок. Если указать ASC, то строки будут выдаваться в порядке возрастания значений сортируемых полей, если DESC

Как видно из синтаксиса оператора **SELECT**, обязательными являются только предложение **SELECT** с перечнем выдаваемых колонок и предложение **FROM**.

Пример простейшего оператора SELECT:

-- Выдать перечень всех служащих:

SELECT * FROM Employee;

Ниже приведено несколько упрощенных вариантов синтаксиса оператора **SELECT**, помогающих научиться составлять простые запросы.

Упрощенный синтаксис внутреннего соединения (стандарт SQL-92):

```
SELECT Колонка [, Колонка ...] | *  
FROM <tableref_left> [INNER] JOIN <tableref_right>  
[ON <условие поиска>]  
[WHERE <условие поиска>];
```

Упрощенный синтаксис внешнего соединения:

```
SELECT Колонка [, Колонка ...] | *  
FROM <tableref_left>  
{LEFT | RIGHT | FULL} [OUTER] JOIN  
<tableref_right>  
[ON <условие поиска>]  
[WHERE <условие поиска>];
```

Упрощенный синтаксис использования подзапроса:

```
SELECT [DISTINCT] Колонка [, Колонка ...]  
FROM <tableref> [, <tableref> ...]  
WHERE  
{expression {[NOT] IN | <оператор сравнения>}  
| [NOT] EXISTS  
}  
(SELECT [DISTINCT] Колонка [, Колонка ...]  
FROM <tableref> [, <tableref> ...]  
WHERE <условие поиска>  
);
```

5.5. Задание

Практическую работу следует выполнять в следующем порядке:

1. Изучить синтаксис оператора **SELECT** и примеры запросов к учебной базе данных ‘**University.mdf**’.

2. Выполнить в окне ‘SQL Editor’ 27 запроса к базе данных, согласно приведенным в практической работе образцам выполнения запросов и сохранять каждый под именами ‘**Lab5-k.sql**’, где k – номер запроса по порядку, в своей рабочей папке. Каждый запрос должен иметь комментарии с описанием, а файл в целом должен иметь комментарии со сведениями об авторе и дате создания.

5.6. Ход работы

Примечание. У вас должны быть перед выполнением этой практической работы созданы все таблицы базы данных университета, созданы ключи, а также заполнены данными.

Выполнение sql-запросов

Для выполнения запросов **SELECT** в программе ‘SQL Server Managment Studio’ необходимо выполнить следующие действия:

1. Подключиться к базе данных и выполнить команду ‘Создать запрос’. В результате откроется окно ‘Конструктора запросов’ (рис. 1).

Далее будут рассмотрены основные выражения, допустимые для условия во фразе WHERE.

Использование простейших условий

Простейшими считаются условия, в которых используются операторы сравнения и логические операторы.

Хотя такие условия являются простейшими в смысле семантики конструкций, они могут иметь довольно сложную структуру из многих операторов сравнений и вложенных друг в друга логических связок.

Операторы сравнения

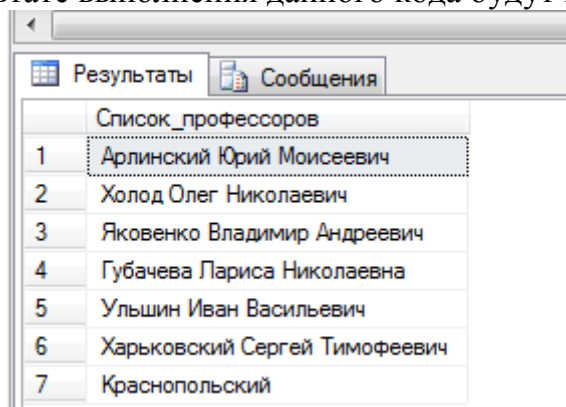
Особенностью операторов сравнения является то, что независимо от типов операндов их результатом являются логические значения. Предположим, вы хотите получить список всех профессоров.

Создайте новый запрос, введите sql-запрос, выполните его, сохраните его в рабочую папку ЛАБ6_SQL под именем 1.sql.

Запрос 1. Вывести фамилии профессоров.

```
SELECT NAME_TEACHER AS 'Список профессоров'  
FROM TEACHER  
WHERE DOLGNOST = 'профессор';
```

Чтобы выполнить sql-команду нажмите на панели редактора кнопку . В результате выполнения данного кода будут выданы все профессора. Например,



Список_профессоров	
1	Арлинский Юрий Моисеевич
2	Холод Олег Николаевич
3	Яковенко Владимир Андреевич
4	Губачева Париса Николаевна
5	Ульшин Иван Васильевич
6	Харьковский Сергей Тимофеевич
7	Краснопольский

Слово 'профессор' в запросе является строковой константой, поэтому ее следует заключить в кавычки. Обратите внимание, что мы указали фразу SELECT без ключевого слова DISTINCT, так как тогда от нас была бы скрыта информация о существовании среди профессоров однофамильцев. Чтобы при выводе результирующий столбец имел содержательный заголовок, мы поименовали его как Список профессоров.

Это первый пример использования предиката над строковым типом данных. Здесь столбец строкового типа сравнивается со строковой константой. Запрос выполнен правильно, однако нужно всегда помнить о том, что предикаты над строками являются чувствительными к регистру букв. Например, предикат 'ИВАНОВ' = 'Иванов' будет ложным. Поэтому, если для некоторого профессора его должность была введена в таблицу TEACHER как 'Профессор', он не будет найден по условию WHERE DOLGNOST = 'профессор'.

Чтобы на предикаты над строками не влиял регистр букв, нужно использовать обычно имеющиеся в СУБД функции преобразования букв в прописные и строчные. В стандарте SQL, например, указаны функции UPPER и LOWER, выполняющие такие преобразования. Следовательно, для предыдущего запроса правильной будет записать

условие фразы WHERE одним из следующих способов:

```
WHERE LOWER(DOLGNOST) = 'профессор'  
WHERE UPPER(DOLGNOST) = 'ПРОФЕССОР'
```

Самостоятельно измените в исходном запросе строку условия с использованием функции изменения регистра.

Чтобы сохранить запрос нажмите правую кнопку и из контекстного меню выберите Сохранить в файл. Присвойте имя 1.sql и сохраните в папку ЛАБ5_SQL.

Запрос 2. Найти всех студентов с стипендией, превышающим 300 грн.

В sql-редакторе создайте новый запрос и введите следующий код:

```
SELECT SUTNAME, SUTFNAME  
FROM STUDENT  
WHERE STIPEND > 300;
```

Выполните его.

Приведем несколько примеров использования операторов сравнения для столбцов строкового и временного типа. Обратите внимание, что сравнивать можно не только значение столбца с константой, но и значения столбцов между собой.

Запрос 3. Вывести фамилии и должности преподавателей, принятых на работу после 01.01.2002.

```
SELECT NAME_TEACHER AS 'Фамилия', DOLGNOST AS 'Должность'  
FROM TEACHER  
WHERE DATA_HIRE > '1/01/2002';
```

Запрос 4. Вывести фамилии и должности преподавателей, фамилии которых в алфавитном порядке располагаются после фамилии Сычева.

```
SELECT NAME_TEACHER, DOLGNOST  
FROM TEACHER  
WHERE UPPER(NAME_TEACHER) > 'Сычева';
```

Самостоятельно создайте запрос 5. Вывести фамилии преподавателей, у которых надбавка меньше ставки в 2,5 и более раз.

Логические операторы

Операндами и результатом логических операторов являются логические значения.

Стандартными логическими операторами являются **AND**, **OR** и **NOT**. Их действие показано в так называемых истинностных таблицах. Использование операторов сравнения вместе с логическими операторами предоставляет возможность формулировать составные условия для отбора строк таблиц.

Использование логического оператора AND

Логический оператор **AND** во многих случаях действует как связка 'и' в русском языке. Рассмотрим несколько примеров с использованием этого оператора.

Запрос 6. Вывести фамилии студентов, проживающих в городе Макарово и имеющих стипендию больше 100 грн.

```
SELECT SUTFNAME  
FROM STUDENT  
WHERE CITY = 'Макарово' AND STIPEND >100;
```

Самостоятельно создайте запрос 7. Вывести фамилии преподавателей, которые являются профессорами и ставка которых превышает 4500.

Самостоятельно создайте запрос 8. Вывести фамилии студентов учащихся на кафедре под порядковым номером 2 (Прикладная математика) с стипендией в диапазоне 100-500 грн.

Использование логического оператора OR

Логический оператор OR во многих случаях действует как связка 'или' в русском языке. Рассмотрим несколько примеров.

Запрос 9. Вывести названия кафедр, расположенных либо в 1 либо в 8 корпусе.

```
SELECT NAME_KAFEDRU, NUM_KORPUSA  
FROM KAFEDRA  
WHERE NUM_KORPUSA =1 OR NUM_KORPUSA =8;
```

Использование логического оператора NOT

Логический оператор NOT в русском языке передается словами 'не' и 'кроме'.

Запрос 10. Вывести названия всех факультетов, кроме факультета математики и информатики.

```
SELECT NAME_FACULTETA  
FROM FACULTET  
WHERE NOT LOWER(NAME_FACULTETA) = 'математики и информатики';
```

Обратите внимание, что оператор NOT должен предшествовать выражению сравнения, а не ставиться перед оператором сравнения. То есть запись LOWER(NAME_FACULTETA) NOT ="математики и информатики" будет неверной. Учитывая, что отрицанием оператора = является оператор <>, вместо указанного условия можно было бы записать LOWER(NAME_FACULTETA) <> "математики и информатики". Это относится ко всем операторам сравнения, так как каждый из них имеет оператор, являющийся его отрицанием.

Комбинирование логических операторов

Логические операторы можно объединять, формируя составные условия. Возможность комбинирования обеспечивается тем, что любой логический оператор возвращает истинностное значение, а значит, его результат может использоваться в другом логическом операторе. Рассмотрим несколько примеров.

Запрос 11. Вывести фамилии, должность, ставку и надбавку ассистентов, у которых либо ставка меньше 550, либо надбавка больше 60.

```
SELECT NAME_TEACHER, DOLGNOST, Salary, Rise  
FROM TEACHER  
WHERE LOWER(DOLGNOST) ='ассистент' AND  
(Salary < 550 OR Rise > 60);
```


Использование выражений над столбцами

До сих пор в выражениях фразы WHERE мы использовали в качестве значения одного или обоих аргументов имена столбцов. Однако аргументами могут быть и выражения над столбцами.

Запрос 12. Показать фамилии преподавателей, чья зарплата (ставка плюс надбавка) превышает 3500.

```
SELECT NAME_TEACHER AS 'Фамилия преподавателя',  
       Salary + Rise AS 'Его зарплата'  
FROM TEACHER  
WHERE Salary + Rise > 3500;
```

Запрос 13. Показать фамилии преподавателей, половина зарплат которых превышает пятикратную надбавку.

```
SELECT NAME_TEACHER  
FROM TEACHER  
WHERE (Salary + Rise) / 2 > 5 * Rise;
```

Использование специальных операторов

В SQL имеются операторы сравнения, позволяющие проверять значения столбцов и выражений над ними на соответствие некоторым специальным условиям:

- принадлежность множеству;
- принадлежность диапазону;
- соответствие шаблону;
- соответствие регулярному выражению;
- неопределенное значение.

В этом разделе вы узнаете, как их использовать и как с их помощью создавать составные условия.

Проверка на принадлежность множеству

Оператор IN позволяет проверить, входит ли значение в указанное множество значений. В простейшем случае этот оператор имеет следующий синтаксис:

имя_столбца [NOT] IN (список_значений)

Здесь список значений представляет собой перечень разделенных запятыми констант, тип которых должен соответствовать типу столбца, чье имя приведено слева. Семантика этого предиката такова: он принимает значение TRUE, если значение столбца соответствует одной из констант списка. Приведем пример.

Запрос 14. Вывести названия и номер корпуса кафедр, расположенных в корпусах 1, 3, 12.

```
SELECT Name_Kafedru, NUM_KORPUSA  
FROM KAFEDRA  
WHERE NUM_KORPUSA IN ('1', '3', '12');
```

Использование отрицания

Так как предикат IN возвращает истинностное значение, к нему можно применить логическое отрицание. Для этого следует воспользоваться нотацией NOT IN. В этом случае предикат будет истинным, если значение столбца не входит в указанный список.

Запрос 15. Вывести названия и номер корпуса кафедр, расположенных в любых корпусах, кроме 1, 3, или 12.

```
SELECT Name_Kafedru AS 'Название кафедры',  
      NUM_KORPUSA AS "Корпус"  
FROM   KAFEDRA  
WHERE  NUM_KORPUSA NOT IN ('1', '3', '12');
```

Использование выражений над столбцами

В левой части оператора IN вместо имени столбца можно использовать любое допустимое над столбцами таблицы выражение языка.

Запрос 16. Вывести фамилии преподавателей, зарплата которых (ставка + надбавка) равна 800, 900, 1000, 1100 или 1200.

```
SELECT NAME_TEACHER AS 'Фамилия преподавателя',  
      Salary + Rise AS 'Зарплата преподавателя'  
FROM   TEACHER  
WHERE  Salary + Rise IN (1150, 2400, 3150, 4300);
```

Более того, элементами списка в правой части оператора IN тоже могут быть выражения над столбцами, как это показано в следующем примере:

Запрос 17.

```
SELECT NAME_TEACHER, Salary, Salary + Rise  
FROM   TEACHER  
WHERE  Salary + Rise IN (Salary + 100, Salary + 200, Salary + 300, Salary + 400,  
Salary + 500);
```

Проверка на принадлежность диапазону значений

Еще одной формой проверки вхождения элемента во множество является проверка на его принадлежность диапазону значений. Для этого применяется предикат BETWEEN, который определяет нахождение значения столбца между указанными минимальным и максимальным значениями. Синтаксис предиката следующий:

имя_столбца [NOT] BETWEEN минимум AND максимум

Проверять можно значения числовых, строковых и временных типов (для строк символов предполагается алфавитное упорядочение). Оператор BETWEEN является включающим - это означает, что крайние значения диапазона включаются в допустимые.

Запрос 18. Вывести фамилии преподавателей со ставкой в диапазоне 1000-2000.

```
SELECT NAME_TEACHER  
FROM   TEACHER  
WHERE  Salary BETWEEN 1000 AND 2000;
```

Использование строковых значений

Использование в операторе BETWEEN в качестве границ диапазона строковых значений имеет особенности, связанные с упорядочением (это же относится и к другим операторам сравнения).

Запрос 19. Вывести фамилии преподавателей, начинающиеся на буквы от 'З' до 'Л'.

```
SELECT NAME_TEACHER  
FROM   TEACHER
```

WHERE UPPER(NAME_TEACHER) BETWEEN 'З' AND 'Л';

Среди строк результата нет фамилий, начинающихся на букву 'Л'. Дело в том, что при сравнении строк символов разной длины SQL предварительно дополняет более короткую строку символами пробела, а он в упорядочениях символов предшествует всем остальным. Поэтому строка, состоящая из буквы 'Л' (дополненная пробелами), всегда будет меньше любой другой строки, в которой за начальной буквой 'Л' следуют отличающиеся от пробела символы.

Чтобы это учесть, в качестве верхнего значения диапазона лучше всего указывать следующую по алфавиту букву (в данном случае — 'М').

Использование отрицания

Так как предикат BETWEEN возвращает истинностное значение, к нему можно применить логическое отрицание. Для этого следует воспользоваться нотацией NOT BETWEEN, в которой предикат будет истинным, только если значение столбца не входит в указанный диапазон. Представление отрицания нотацией NOT BETWEEN введено в язык для большей наглядности, так как с предикатом BETWEEN можно стандартным образом использовать логический оператор NOT (то есть ставить отрицание ко всему выражению, а не к предикату):

NOT (имя_столбца BETWEEN минимум AND максимум)

Круглые скобки в данном случае можно и опустить, так как они не меняют порядка исполнения операторов. В нотации NOT BETWEEN крайние значения в диапазон не включаются.

Запрос 20. Вывести названия и номер корпуса кафедр, которые не расположены в корпусах 1 и 3.

```
SELECT Name_Kafedru, NUM_KORPUSA  
FROM KAFEDRA  
WHERE NUM_KORPUSA NOT BETWEEN '1' AND '3';
```

```
SELECT Name_Kafedru, NUM_KORPUSA  
FROM KAFEDRA  
WHERE NOT (NUM_KORPUSA BETWEEN '1' AND '3');
```

Использование выражений над столбцами

Как и в предикате IN, вместо имени столбца и границ диапазона можно использовать любое допустимое в языке выражение над столбцами таблицы, включая и функции.

Запрос 21. Показать фамилии преподавателей, принятых на работу между 01.01.2000 и 12.12.2001.

```
SELECT NAME_TEACHER, DATA_HIRE  
FROM TEACHER  
WHERE DATA_HIRE BETWEEN '01/01/2000' AND '12/12/2001';
```

Запрос 22. Вывести данные преподавателей, зарплата которых (ставка + надбавка) находится в диапазоне от удвоенной величины надбавки до утроенной надбавки плюс 50.

```
SELECT NAME_TEACHER, Salary + Rise, 2 * Rise, 3 * Rise + 50  
FROM TEACHER  
WHERE Salary + Rise BETWEEN 2 * Rise AND 3 * Rise + 50;
```

Проверка на соответствие шаблону

Когда необходимо отобрать строки таблицы, в которых значение некоторого столбца совпадает с заданной строкой символов, следует использовать обычное сравнение, как это показано выше. Однако во многих случаях можно не знать точное представление в базе данных интересующего значения. Название одной и той же кафедры, например, может храниться в одном из следующих вариантов: 'базы данных', 'организация баз данных', 'информационные системы и базы данных', 'базы данных и знаний'.

Такая же ситуация возникает, когда не известно точное написание фамилии преподавателя, название дисциплины, факультета и т. п. Специально для таких случаев предназначен оператор сравнения LIKE, позволяющий отобрать из таблицы строки на основе частичного соответствия. Упрощенный синтаксис оператора следующий:

имя_столбца [NOT] LIKE шаблон [ESCAPE символ_пропуска]

Его можно использовать только с символьными значениями.

Использование шаблона

Оператор LIKE сравнивает значение столбца с множеством значений, определяемых шаблоном. Он представляет собой строку, в которой помимо обычных символов, составляющих основу поискового выражения, можно использовать так называемые подстановочные символы (иногда они называются групповыми символами). Имеется всего два подстановочных символа, различающихся тем, что именно на их месте может стоять:

% — любая последовательность символов, включая их отсутствие;

_ — один любой символ.

Подстановочные символы могут находиться в любом месте шаблона в любом наборе.

Например, шаблону '%Иван%' соответствуют строки 'Иван', 'Иванов', 'Иванченко', 'Петр Иванович', а шаблону 'л_с_' - 'лист', 'леса', 'лоск' (ноне 'лес', 'листок', 'плес').

Оператор LIKE, как и все другие, работающие с символьными строками, чувствителен к регистру букв, поэтому при его использовании мы рекомендуем использовать уже известные вам функции UPPER() и LOWER().

Запрос 23. Найти фамилии преподавателей на букву 'М'.

```
SELECT NAME_TEACHER  
FROM TEACHER  
WHERE UPPER(NAME_TEACHER) LIKE 'M%';
```

Имейте в виду, что если вы запишете условие фразы WHERE как UPPER(NAME_TEACHER) = 'M%' или даже как 'M%' LIKE UPPER(NAME_TEACHER), фамилии преподавателей будут сравниваться со строкой ' M%'. Во втором случае выражение является синтаксически правильным оператором LIKE, однако в нем строка 'M%' не выступает в качестве шаблона, так как расположена перед ключевым словом LIKE.

Запрос 24. Указать преподавателей, в фамилиях которых первой буквой является 'М', а четвертой – 'ы'.

```
SELECT NAME_TEACHER  
FROM TEACHER
```

WHERE NAME_TEACHER LIKE 'М__ ы%';

Запрос 25. Вывести названия кафедр, в которых присутствует словосочетание 'анализ' (в различных грамматических формах).

```
SELECT Name_Kafedru  
FROM KAFEDRA  
WHERE LOWER(Name_Kafedru) LIKE '%анализ%';
```

В левой части оператора LIKE может находиться не только имя столбца, но и любое допустимое над столбцами выражение, как это показано в следующем примере.

Запрос 26. Указать преподавателей, в фамилию и название должности которых входит в сумме не меньше пяти букв 'о'.

```
SELECT NAME_TEACHER, DOLGNOST  
FROM TEACHER  
WHERE LOWER(NAME_TEACHER + DOLGNOST) LIKE  
'%о%о%о%о%о%';
```

Проверка на неопределенное значение

Как мы уже отмечали, наличие значения NULL во фразе WHERE приводит к тому, что условие принимает истинностное значение UNKNOWN и соответствующая строка не включается в результат. Детальное описание работы с неопределенным значением вы можете найти в уроке 10, а здесь мы покажем, как обрабатывать значение NULL во фразе WHERE.

Чтобы проверить столбец на неопределенное значение, следует применить унарный оператор IS NULL, имеющий такой синтаксис:

имя_столбца IS [NOT] NULL

Этот оператор принимает истинностное значение TRUE, если столбец имеет неопределенное значение, и FALSE — в противном случае. В нотации IS NOT NULL его действие обратное.

Запрос 27. Вывести фамилии преподавателей, у которых не задан номер телефона или идентификационный код.

```
SELECT NAME_TEACHER, INDEF_KOD, TEL_TEACHER  
FROM TEACHER  
WHERE INDEF_KOD IS NULL OR TEL_TEACHER IS NULL;
```

Задание для практической работы №5

Для созданной базы данных, согласно номеру варианта, самостоятельно создать **на языке Transact-SQL 15 запросов с отбором строк по условию:**

- 3 простейших запроса с использованием операторов сравнения;
- 3 запроса с использованием логических операторов AND, OR и NOT;
- 1 запрос на использование комбинации логических операторов;
- 1 запрос на использование выражений над столбцами;
- 2 запроса с проверкой на принадлежность множеству;
- 2 запроса с проверкой на принадлежность диапазону значений;
- 2 запроса с проверкой на соответствие шаблону;
- 1 запрос с проверкой на неопределенное значение.

Все программные инструкции команд SQL сохранять в файлах с расширением *.sql в папке **ФИО_студента/Лаб5**.

Для каждого запроса сформулировать текстовое задание, которое должно быть выполнено к базе данных.

Создать текстовый отчет, в котором отобразить sql-команды разработанных запросов и скриншоты результатов работы из СУБД **SQL Server Management Studio**.

ПРАКТИЧЕСКАЯ РАБОТА №6. СОЗДАНИЕ МНОГОТАБЛИЧНЫХ ЗАПРОСОВ. ЗАПРОСЫ НА СОЕДИНЕНИЕ

6.1. Цель практической работы

Изучить используемый в реляционных СУБД оператор извлечения данных из таблиц. Получить навыки работы с оператором SELECT в программе 'SQL Server Managment Studio'.

6.2. Исходные данные

Исходными данными является индивидуальное задание и результат предыдущих практических работ.

6.3. Используемые программы

Программы 'SQL Server Managment Studio'.

6.4. Теоретические сведения

При проектировании стремятся создавать таблицы, в каждой из которых содержалась бы информация об одном и только одном типе сущности. Это облегчает модификацию базы данных и поддержку ее целостности. Именно так мы поступили, создавая учебную базу данных. Однако сущности могут быть взаимосвязанными. Кафедры связаны с факультетами по признаку вхождения в их состав, преподаватели работают на кафедрах, студенты учатся на кафедрах и т. д.

Связь между таблицами устанавливается за счет размещения специального столбца первичного ключа одной таблицы, которая называется родительской, в другой таблице, которая называется дочерней. Столбец (или совокупность столбцов) дочерней таблицы, определенный для связи с родительской таблицей, называется внешним ключом.

Наличие внешних ключей является основой для инициирования поиска по многим таблицам.

Одна из наиболее важных особенностей предложения SELECT — это способность использования связей между различными таблицами, а также вывода содержащейся в них информации. Операция, которая приводит к соединению из двух таблиц всех пар строк, для которых выполняется заданное условие, называется соединением таблиц. Для того чтобы указать соединяемые таблицы, их следует перечислить через запятую во фразе FROM.

Декартово произведение таблиц

Соединение таблиц - это частный случай операции декартового произведения (или просто произведения). Декартово произведение двух таблиц — это таблица, состоящая из всех возможных пар строк обеих таблиц. Это определение можно естественным образом расширить на любое количество таблиц. В SQL декартово произведение выражается указанием имен перемножаемых таблиц во фразе FROM и указанием всех их столбцов во фразе SELECT.

Так, произведение таблиц FACULTET и KAFEDRA выражается следующим образом:

```
SELECT *  
FROM FACULTET, KAFEDRA
```

Так как результирующая таблица содержит много столбцов и они не помещаются по ширине страницы, мы приведем только интересующие нас столбцы произведения этих таблиц.

Запрос 28. Декартово произведение таблиц.

```
SELECT FACULTET.Name_faculteta, FACULTET. Kod_faculteta,  
      KAFEDRA. Kod_faculteta, KAFEDRA.Name_Kafedru  
FROM  FACULTET, KAFEDRA;
```

Каждая строка таблицы факультетов оказалась соединенной с каждой строкой таблицы кафедр, в результате получилось 27 строк (3 факультета x 9 кафедр = 27 комбинаций).

В произведении может участвовать много таблиц. Например, произведение таблиц факультетов, кафедр и преподавателей записывается следующим образом:

```
SELECT *  
FROM  FACULTET, KAFEDRA, TEACHER
```

Условие соединения

Соединение таблиц может быть указано во фразе WHERE или во фразе FROM. Сначала рассмотрим первый вариант. Большинство запросов, имеющих несколько таблиц во фразе FROM, содержат фразу WHERE, в которой указаны условия, попарно сравнивающие столбцы из различных таблиц. Такое условие называется условием соединения. В этом случае SQL предполагает сцепление только тех пар строк из разных таблиц, для которых условие соединения принимает истинное значение. Теоретически при соединении сначала выполняется декартово произведение указанных таблиц в одну, а затем из нее отбираются строки согласно условию соединения. Естественно, ни одна СУБД не работает таким образом.

Фраза WHERE помимо условия соединения может также содержать другие условия, каждое из которых ссылается на столбцы соединенной таблицы. Эти условия производят отбор строк соединенной таблицы.

Соединения можно разделить на следующие категории.

Внутренние соединения (типичные операции соединения, использующие такие операторы сравнения, как = или <>). Они включают эквивалентные соединения и естественные соединения.

Внутренние соединения используют оператор сравнения для установки соответствия строк из двух таблиц на основе значений общих столбцов в каждой таблице. Примером может быть получение всех строк, в которых идентификационный номер студента одинаковый как в таблице students, так и в таблице courses.

Внешние соединения. Внешние соединения бывают левыми, правыми и полными.

Если внешние соединения задаются в предложении FROM, они указываются с одним из следующих наборов ключевых слов.

LEFT JOIN или LEFT OUTER JOIN

Результирующий набор левого внешнего соединения включает все строки из левой таблицы, заданной в предложении LEFT OUTER, а не только те, в которых соединяемые столбцы соответствуют друг другу. Если строка в левой таблице не имеет совпадающей строки в правой таблице, результирующий набор строк содержит значения NULL для всех столбцов списка выбора из правой таблицы.

RIGHT JOIN или RIGHT OUTER JOIN

Правое внешнее соединение является обратным для левого внешнего соединения. Возвращаются все строки правой таблицы. Для левой таблицы возвращаются значения NULL каждый раз, когда строка правой таблицы не имеет совпадающей строки в левой таблице.

FULL JOIN или FULL OUTER JOIN

Полное внешнее соединение возвращает все строки из правой и левой таблицы. Каждый раз, когда строка не имеет соответствия в другой таблице, столбцы списка выбора другой таблицы содержат значения NULL. Если между таблицами имеется соответствие, вся строка результирующего набора содержит значения данных из базовых таблиц.

Перекрестные с соединения

Перекрестное соединение возвращает все строки из левой таблицы. Каждая строка из левой таблицы соединяется со всеми строками из правой таблицы. Перекрестные соединения называются также декартовым произведением.

Таблицы или представления в предложении FROM могут указываться в любом порядке с внутренним соединением или полным внешним соединением. Однако важен порядок таблиц или представлений, заданных при использовании левого или правого внешнего соединения.

Соединение таблиц по равенству

Если таблицы соединяются по равенству значений пары столбцов (группы столбцов) из различных таблиц, такая операция называется соединением таблиц по равенству. Соединение по равенству, в отличие от декартового произведения, позволяет соединить только те пары строк, которые действительно взаимосвязаны друг с другом. Так, например, мы можем соединить таблицы факультетов и кафедр по условию FACULTET.Kod_faculteta = KAFEDRA.Kod_faculteta. В таком варианте мы соединяем таблицы осмысленно, так как каждая строка таблицы FACULTET соединяется только со строками соответствующих кафедр. На базе таблиц FACULTET и KAFEDRA мы получаем таблицу со столбцами из обеих таблиц, имеющую строки с понятным смыслом. Можно также сказать, что в таблицу KAFEDRA вместо столбца Kod_faculteta мы вставляем все характеристики (столбцы) соответствующего факультета из таблицы FACULTET.

Соединение таблиц используется, когда необходимо вывести значения столбцов:

- разных таблиц;
- одной таблицы, но отвечающих условию, заданному на другой таблице.

Эти два варианта, а также их комбинация, характерны для любого вида соединения, а не только по равенству. Перейдем к рассмотрению примеров.

Вывод столбцов разных таблиц

Этот вид запросов характерен тем, что фраза WHERE содержит только условие соединения, а список фразы SELECT содержит имена столбцов из различных таблиц.

Запрос 29. Вывести названия кафедр и номера их групп.

```
SELECT Name_Kafedru, [Group]  
FROM KAFEDRA, STUDENT  
WHERE KAFEDRA.kod_kafedru = STUDENT.kod_kafedru;
```

или

```
SELECT Name_Kafedru, student.[GROUP]  
FROM KAFEDRA, STUDENT  
WHERE KAFEDRA.kod_kafedru = STUDENT.kod_kafedru;
```

Мы привели два варианта запроса. В первом имена столбцов не уточняются именами таблиц, а во втором — уточняются. В данном случае это не имеет значения, оба запроса корректны.

Уточнение имен столбцов

До тех пор, пока запрос относится к одной таблице, обращение к столбцам по их именам не вызывает проблем — в таблице все имена столбцов должны быть неповторяющимися. Однако как только запрос соединяет несколько таблиц, может возникнуть неоднозначность при ссылках на столбцы с одинаковыми именами из разных таблиц. Для разрешения этой неоднозначности во фразах SELECT и WHERE (как и в некоторых других фразах) имена столбцов необходимо уточнять именами таблиц.

Запрос 30. Вывести названия факультетов и их кафедр.

```
SELECT FACULTET.NAME_FACULTETA, KAFEDRA.Name_Kafedru  
FROM FACULTET, KAFEDRA  
WHERE FACULTET.Kod_faculteta = KAFEDRA.Kod_faculteta;
```

В этом запросе мы уточнили имена столбцов во фразах SELECT и WHERE, хотя во втором случае это не обязательно, так как используются неповторяющиеся имена. Тем не менее, рекомендуем при соединении таблиц для наглядности уточнять имена столбцов. Обратите внимание на то, что в предыдущем примере отсутствует факультет математики — на нем нет кафедр.

Вывод столбцов с условием отбора

Вариант, когда отбираются строки одной таблицы, а условие задается с участием другой, используется довольно часто. Приведем примеры.

Запрос 31. Вывести названия кафедр факультета Математики и информатики.

```
SELECT KAFEDRA.Name_Kafedru AS 'Кафедры факультета математики и  
информатики'  
FROM FACULTET, KAFEDRA  
WHERE FACULTET.Kod_faculteta = KAFEDRA.Kod_faculteta AND  
LOWER(FACULTET.NAME_FACULTETA) = 'математики и информатики';
```

Запрос 32. Вывести фамилии доцентов кафедры информатики.

```
SELECT TEACHER.NAME_TEACHER AS 'Доценты кафедры информатики'  
FROM KAFEDRA, TEACHER  
WHERE KAFEDRA.kod_kafedru = TEACHER.kod_kafedru AND  
LOWER(KAFEDRA.Name_Kafedru) = 'информатики' AND  
LOWER(TEACHER.DOLGNOST) = 'доцент';
```

В последнем запросе помимо условия соединения используется также отбор строк по условиям, заданным для разных таблиц.

Синонимы таблиц

Синонимы таблиц часто используются для задания более лаконичного имени таблицы, по которому можно сослаться на нее в любых других местах запроса. Приведем пример.

Запрос 33. Вывести названия кафедр, на которых имеются студенты со стипендией >200 грн.

```
SELECT DISTINCT k.Name_Kafedru  
FROM KAFEDRA k, STUDENT s  
WHERE k.Kod_kafedru = s. Kod_kafedru AND s.Stipend > 400;
```

Запросы по трем и более таблицам

SQL позволяет формулировать запросы, которые предполагают использование трех и более таблиц. При этом следует применять ту же методику соединения, что и для двух таблиц. Рассмотрим простой пример соединения трех таблиц.

Запрос 34. Вывести названия тех кафедр факультета математики и информатики, на которых работают профессора.

```
SELECT DISTINCT KAFEDRA.Name_Kafedru  
FROM FACULTET, KAFEDRA, TEACHER  
WHERE FACULTET.Kod_faculteta = KAFEDRA.Kod_faculteta AND  
KAFEDRA.Kod_kafedru = TEACHER.Kod_kafedru AND  
FACULTET.Name_faculteta = 'Математики и информатики' AND  
TEACHER.DOLGNOST = 'профессор';
```

Для ответа на запрос необходимы три таблицы: на таблицах факультетов и преподавателей заданы условия отбора, а из таблицы кафедр следует вывести столбец названий. Поэтому три необходимые таблицы указываются во фразе FROM, а во фразе WHERE производится их соединение по условию равенства первичного и внешнего ключей:

FACULTET. Kod_faculteta = KAFEDRA. Kod_faculteta -- соединение таблиц факультетов и кафедр

KAFEDRA. Kod_kafedru = TEACHER. Kod_kafedru -- соединение таблиц кафедр и преподавателей

Таблица, образуемая в результате соединений, будет иметь столько же строк, сколько имеется в таблице преподавателей (если все преподаватели работают на кафедрах). Выясним, почему это так, но сначала заметим, что результат соединения таблиц не зависит от порядка соединения. Поэтому рассмотрим случай, когда сначала мы соединяем таблицы кафедр и преподавателей, а затем результат соединяем с таблицей факультетов.

Так как между таблицами кафедр и преподавателей существует связь типа один-ко-многим, их соединение фактически означает приписывание к строке каждого преподавателя данных о его кафедрах. Количество строк этого соединения будет равным количеству преподавателей. Связь между таблицами факультетов и кафедр также имеет тип один-ко-многим, поэтому второе соединение означает, что к каждой строке таблицы, полученной после первого соединения, приписываются данные о факультете кафедры. Таким образом, количество строк останется равным числу преподавателей.

Вернемся к запросу. Последние два условия фразы WHERE отбирают строки из соединенной таблицы, а во фразе SELECT указан выводимый столбец. Ключевое слово DISTINCT указано в нем потому, что названия кафедр в соединенной таблице могут повторяться.

Запрос 35. Вывести фамилии ассистентов факультета математики и информатики.

```
SELECT TEACHER.NAME_TEACHER AS 'Ассистенты ф-та математики и информатики'  
FROM FACULTET, KAFEDRA, TEACHER  
WHERE FACULTET.Kod_faculteta = KAFEDRA.Kod_faculteta AND  
KAFEDRA.Kod_kafedru = TEACHER.Kod_kafedru AND  
FACULTET.Name_faculteta = 'Математики и информатики' AND  
TEACHER.DOLGNOST = 'ассистент';
```

В этом случае для ответа нужны две таблицы — факультетов и преподавателей. Однако они связаны между собой опосредованно, через таблицу кафедр. Поэтому для соединения таблиц факультетов и преподавателей следует использовать таблицу кафедр.

Сформулируем общую процедуру составления многотабличного запроса и приведем пример ее использования.

1. Определить множество таблиц, необходимых для ответа на запрос. В это множество должны входить таблицы, на столбцах которых сформулированы условия, а также те, столбцы которых необходимо вывести. Это так называемые базовые таблицы запроса.

2. В структуре взаимосвязанных таблиц найти путь, соединяющий базовые таблицы. Это так называемый путь вычисления запроса. В результате вы получите перечень таблиц, необходимых для формулировки запроса. Это так называемые таблицы запроса.

3. Во фразе FROM перечислить необходимые таблицы.

4. Во фразе WHERE соединить таблицы запроса и при необходимости задать условия отбора строк в базовых таблицах запроса.

5. Во фразе SELECT перечислить выводимые столбцы.

Вывод всех столбцов соединяемой таблицы

В многотабличном запросе конструкция SELECT * означает выбор всех столбцов соединенной таблицы. Например, результирующая таблица следующего запроса состоит из 21 столбца: 5 столбцов таблицы факультетов, 6 столбцов таблицы кафедр и 10 столбцов таблицы преподавателей.

Запрос 36.

```
SELECT *  
FROM FACULTET f, KAFEDRA k, TEACHER t  
WHERE f.Kod_faculteta = k.Kod_faculteta AND k.Kod_kafedru =  
t.Kod_kafedru;
```

При наличии в запросе многих таблиц конструкция SELECT * становится не очень практичной. В связи с этим в различных СУБД предоставляется возможность использовать во фразе SELECT многотабличных запросов выражение имя_таблицы.* для указания вывода всех столбцов конкретной таблицы. Например:

```
SELECT f.*, k.FIO_ZAVKAF, t.*  
FROM FACULTET f, KAFEDRA k, TEACHER t  
WHERE f.Kod_faculteta = k.Kod_faculteta AND k.Kod_kafedru =  
t.Kod_kafedru;
```

Другие виды соединений по равенству

Логическая связь между таблицами поддерживается взаимосоответствием столбцов первичного и внешнего ключей. Все рассмотренные до сих пор запросы для соединения таблиц использовали именно эту связь. Однако SQL позволяет связывать таблицы по любой паре столбцов, которые имеют сравнимые типы данных, независимо от того, имеет ли эта связь какой-либо смысл. Рассмотрим ряд примеров.

Запрос 37. Если фамилия заведующего кафедры совпадает с фамилией декана какого-нибудь из факультетов, вывести название этой кафедры вместе с названием соответствующего факультета.

```
SELECT k.Name_Kafedru AS 'Название кафедры',  
       f.NAME_FACULTETA AS 'Название факультета'  
FROM   FACULTET f, KAFEDRA k  
WHERE  f.FIO_DECANA = k.FIO_ZAVKAF;
```

Запрос 38. Вывести пары названий кафедр и фамилий преподавателей, у которых совпадают первичные ключи.

```
SELECT k.Name_Kafedru AS 'Название кафедры',  
       t.Name_Teacher AS 'Фамилия преподавателя'  
FROM   KAFEDRA k, TEACHER t  
WHERE  k.Kod_kafedru = t.Kod_kafedru;
```

Если первый запрос не лишен смысла, то последний абсолютно бессмысленный, так как в учебной базе данных первичные ключи лишены какого-либо содержания и используются только для идентификации строк своих таблиц.

Самосоединение таблицы

Как правило, взаимосвязи существуют и в пределах одной таблицы. В одних случаях эти связи являются явными, например, когда внешний ключ ссылается на первичный ключ той же самой таблицы. В других случаях эта связь присутствует неявно, например, кафедры могут быть связаны между собой на основании того свойства, что располагаются в одном корпусе.

Для ответа на такие запросы следует осуществлять соединение таблицы со своей копией. Такое соединение иногда называют самосоединением таблицы. Несмотря на кажущуюся искусственность идеи самосоединения таблиц, существует множество запросов, которые требуют именно такого соединения. На приводимых далее примерах вы убедитесь в этом.

Чтобы произвести соединение таблицы со своей копией, необходимо указать во фразе FROM имя одной и той же таблицы два или большее количество раз, а во фразе WHERE — условие их самосоединения.

Однако в этом случае возникает следующая проблема — как ссылаться на столбцы различных копий таблицы. До сих пор проблема ссылки на столбцы с одинаковыми именами из разных таблиц разрешалась уточнением имени столбца именем таблицы. В нашем же случае соединяемые таблицы имеют одинаковые имена. Для разрешения этой проблемы без синонимов таблиц уже не обойтись. В нашем случае различным вхождениям одной и той же таблицы приписываются различные синонимы и именно по этим синонимам производится обращение к столбцам. Приведем примеры использования самосоединения.

Запрос 39. Вывести фамилии преподавателей, зарплата которых больше, чем у преподавателя Сидорова.

```
SELECT needed.NAME_TEACHER
FROM TEACHER needed, TEACHER given
WHERE needed.Salary + needed.Rise > given.Salary + given.Rise AND
given.NAME_TEACHER = 'Игнатъева Олеся Владимировна';
```

Симметричное соединение и удаление избыточности

При самосоединении по равенству обычно возникают избыточные строки. Рассмотрим следующий запрос.

Запрос 40. Вывести названия кафедр, располагающихся в том же корпусе, что и кафедра информатики.

```
SELECT needed.Name_Kafedru
FROM KAFEDRA needed, KAFEDRA given
WHERE needed.NUM_KORPUSA = given.NUM_KORPUSA AND
given.Name_Kafedru = 'Информатики';
```

Обратите внимание, что в результат включена и сама кафедра информатики. Для того чтобы избавиться от ненужной результирующей строки, следует добавить условие отбора:

```
needed.Name_Kafedru <> ' Информатика'
```

При самосоединении по равенству можно получить симметричную результирующую таблицу. Суть симметричности заключается в том, что в таблице содержатся строки:

- с одинаковыми значениями всех столбцов;
- со всеми возможными перестановками значений столбцов.

Запрос 41. Вывести пары номеров групп, которые принадлежат одной кафедре.

```
SELECT g1.[Group], g2.[Group], g1.kod_kafedru
FROM STUDENT g1, STUDENT g2
WHERE g1.kod_kafedru = g2.kod_kafedru;
```

ПРИМЕЧАНИЕ В этом запросе мы дополнительно вывели столбец с номером (первичным ключом) кафедры для большей наглядности.

Результирующая таблица оказалась симметричной, и в связи с этим содержит избыточные строки.

Простой способ избежать этого состоит в том, чтобы наложить ограничение на выбираемые пары значений таким образом, чтобы первое выдаваемое значение было меньше другого (или предшествовало ему в алфавитном порядке). Это делает результат асимметричным, поэтому пары с одинаковыми значениями, а также пары, заданные в обратном порядке, не будут появляться. Покажем это на примере варианта предыдущего запроса.

```
SELECT g1.[Group], g2.[Group], g1.kod_kafedru
FROM STUDENT g1, STUDENT g2
WHERE g1.kod_kafedru = g2.kod_kafedru
AND g1.[Group] <g2.[Group];
```

Проверка правильности данных

Самосоединение можно использовать для проверки корректности данных. Например, мы точно знаем, что в нашем вузе нет однофамильцев, занимающих разные должности. С помощью самосоединения таблицы преподавателей мы можем убедиться, что их нет и в базе данных.

Запрос 42. Указать преподавателей-однофамильцев, которые занимают различные должности.

```
SELECT tch1.NAME_TEACHER AS 'Препод. с различ. должностями'  
FROM TEACHER tch1, TEACHER tch2  
WHERE tch1.NAME_TEACHER = tch2.NAME_TEACHER AND  
tch1.DOLGNOST <> tch2.DOLGNOST;
```

Внешнее соединение таблиц

Предположим, необходимо вывести список факультетов и их кафедр. Это достигается соединением таблиц FACULTET и KAFEDRA по равенству значений первичного и внешнего ключей и выбором столбцов с названиями факультетов и кафедр. Но в таком случае, если на факультете кафедр нет, он не будет включен в результат.

Для того чтобы в списке присутствовали все факультеты, даже без кафедр, необходимо использовать внешнее соединение, которое расширяет возможности обычного соединения. Внешнее соединение возвращает строки, которые удовлетворяют условию соединения, а также те строки одной из таблиц, для которых в другой не нашлось удовлетворяющих условию соединения строк.

Внутренние соединения возвращают результат, когда в обеих таблицах есть хотя бы одна строка, соответствующая условиям соединения. Внутренние соединения исключают строки, не соответствующие ни одной строке в другой таблице. Однако внешние соединения возвращают все строки хотя бы из одной таблицы или представления, упомянутых в предложении FROM, если они удовлетворяют условиям поиска WHERE или HAVING.

Все строки, получаемые из левой таблицы, образуют левое внешнее соединение, а строки, получаемые из правой таблицы, — правое внешнее соединение. Все строки их обеих таблиц возвращаются в полном внешнем соединении.

Для внешних соединений в предложении FROM SQL Server использует ключевые слова ISO:

```
LEFT OUTER JOIN или LEFT JOIN;  
RIGHT OUTER JOIN или RIGHT JOIN;  
FULL OUTER JOIN или FULL JOIN.
```

Работа с левыми внешними соединениями

Рассмотрим примеры.

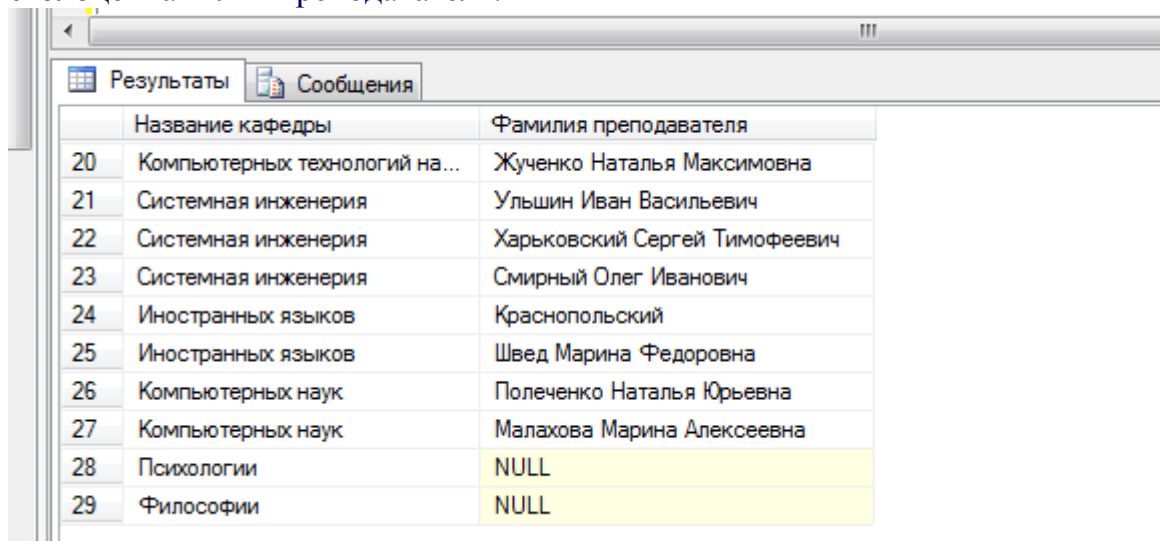
Рассмотрим соединение таблиц KAFEDRA и TEACHER по столбцам kod_kafedru. В результате будут выведены только те кафедры, для которых были написаны преподаватели.

Чтобы включить в результаты все кафедры, независимо от того, были ли написаны их преподаватели, используйте левое внешнее соединение ISO. Пример запроса:

Запрос 43. Вывести фамилии всех преподавателей с указанием их кафедры, если она есть.

```
SELECT KAFEDRA.Name_Kafedru AS 'название кафедры',  
TEACHER.NAME_TEACHER AS 'фамилия преподавателя'  
FROM KAFEDRA LEFT OUTER JOIN TEACHER  
ON KAFEDRA.kod_kafedru = TEACHER.kod_kafedru;
```

Ключевые слова **LEFT OUTER JOIN** включают в вывод все строки таблицы **KAFEDRA** независимо от того, есть ли для них соответствующие значения в столбце **kod_kafedru** таблицы **TEACHER**. Обратите внимание на то, что в результатах, где для кафедры нет соответствующего преподавателя, строки содержат значение **NULL** в столбце **Фамилия преподавателя**.



	Название кафедры	Фамилия преподавателя
20	Компьютерных технологий на...	Жученко Наталья Максимовна
21	Системная инженерия	Ульшин Иван Васильевич
22	Системная инженерия	Харьковский Сергей Тимофеевич
23	Системная инженерия	Смирный Олег Иванович
24	Иностранных языков	Краснопольский
25	Иностранных языков	Швед Марина Федоровна
26	Компьютерных наук	Полеченко Наталья Юрьевна
27	Компьютерных наук	Малахова Марина Алексеевна
28	Психологии	NULL
29	Философии	NULL

Работа с правыми внешними соединениями

Рассмотрим соединение таблиц **KAFEDRA** и **TEACHER** по столбцам **kod_kafedru**. Оператор правого внешнего соединения **ISO**, **RIGHT OUTER JOIN**, включает в результаты все строки второй таблицы независимо от того, есть ли для них совпадающие данные в первой таблице.

Чтобы включить в результаты всех преподавателей независимо от того, есть ли связанные с ними кафедры, используйте правое внешнее соединение **ISO**. Пример запроса **Transact-SQL** и результаты правого внешнего соединения:

Запрос 44. Вывести названия всех кафедр с указанием фамилий преподавателей, если они есть.

```
SELECT KAFEDRA.Name_Kafedru AS 'название кафедры',
TEACHER.NAME_TEACHER AS 'фамилия преподавателя'
FROM KAFEDRA RIGHT OUTER JOIN TEACHER
ON KAFEDRA.kod_kafedru = TEACHER.kod_kafedru;
```

Внешнее соединение и условие отбора

При внешнем соединении можно применять и дополнительные условия отбора строк. Как видно из следующих двух примеров, если условие относится к столбцам таблицы, к которой не применяется оператор внешнего соединения, то внешнее соединение происходит.

Запрос 45. Вывести названия всех кафедр корпуса 1 с указанием их преподавателей, если они есть.

```
SELECT KAFEDRA.Name_Kafedru AS 'название кафедры',
TEACHER.NAME_TEACHER AS 'фамилия преподавателя'
FROM KAFEDRA LEFT OUTER JOIN TEACHER
ON KAFEDRA.kod_kafedru = TEACHER.kod_kafedru
```


WHERE KAFEDRA.NUM_KORPUSA = '1';

Запрос 46. Вывести названия всех кафедр с указанием их преподавателей, если они есть, ставка которых больше 3000.

```
SELECT KAFEDRA.Name_Kafedru AS 'Название кафедры',  
TEACHER.NAME_TEACHER AS 'Фамилия преподавателя'  
FROM KAFEDRA RIGHT OUTER JOIN TEACHER  
ON KAFEDRA.kod_kafedru = TEACHER.kod_kafedru  
WHERE TEACHER.salary > 3000;
```

Работа с полными внешними соединениями

Чтобы сохранить в выводе не соответствующие друг другу строки из обеих таблиц, включив их в результаты соединения, используйте полное внешнее соединение. SQL Server предоставляет оператор полного внешнего соединения, **FULL OUTER JOIN**, включающий все строки из обеих таблиц вне зависимости от того, есть ли в них совпадающие значения.

Использование перекрестных соединений

Перекрестное соединение, не имеющее предложения WHERE, выполняет декартово произведение таблиц, вовлеченных в объединение. Размер результирующего набора декартова произведения вычисляется, как произведение количества строк в первой таблице на количество строк во второй таблице. Следующий пример показывает перекрестное соединение Transact-SQL.

```
SELECT KAFEDRA.Name_Kafedru AS 'название кафедры',  
TEACHER.NAME_TEACHER AS 'фамилия преподавателя'  
FROM KAFEDRA CROSS JOIN TEACHER  
ORDER BY KAFEDRA.kod_kafedru;
```

Результирующий набор содержит 297 строк (в **KAFEDRA** имеется 11 строк, а в таблице **TEACHER** существует 27 строк; 11, умноженное на 27, равно 297).

Внешнее соединение трех и более таблиц

В запросе может быть использовано внешнее соединение более чем двух таблиц.

Хотя в операции соединения указываются всего две таблицы, предложение FROM может содержать несколько операций объединения. Это позволяет соединять в одном запросе несколько таблиц.

При этом следует помнить, что если к столбцу таблицы A применен оператор внешнего соединения с таблицей B, то никакой другой столбец таблицы A не может содержать оператор внешнего соединения с таблицей, отличающейся от B.

В следующем примере внешнее соединение применяется для трех таблиц — факультетов, кафедр и преподавателей.

Запрос 47. Вывести список всех факультетов с указанием их кафедр и преподавателей.

```
SELECT f.NAME_FACULTETA AS 'Факультет',  
k.Name_Kafedru AS 'Кафедра',  
t.NAME_TEACHER AS 'Преподаватель'  
FROM FACULTET f JOIN KAFEDRA k  
ON f.kod_faculteta = k.kod_faculteta  
JOIN TEACHER t  
ON k.kod_kafedru = t.kod_kafedru;
```

Следующий запрос Transact-SQL выполняет поиск наименований всех факультетов определенной кафедры и имени преподавателей этих кафедр.

Обратите внимание, что ни один из соединяемых столбцов — ни `kod_faculteta`, ни `kod_kafedru`, не включается в результаты. Тем не менее, соединение возможно только при использовании `Kafedra` в качестве промежуточной таблицы.

Среднюю таблицу соединения, `Kafedra`, можно назвать таблицей преобразования, или промежуточной таблицей, так как `Kafedra` является промежуточной точкой объединения, которая находится между двумя другими участвующими в объединении таблицами.

При наличии в инструкции нескольких операторов соединения, применяющихся либо при соединении более двух таблиц, либо при соединении более двух пар столбцов, выражения соединения могут быть связаны операторами `AND` или `OR`.

Задание для практической работы №6

Для созданной базы данных, согласно номеру варианта, самостоятельно создать **на языке Transact-SQL 15 многотабличных запросов**:

- 1 запрос с использованием декартового произведения двух таблиц;
- 3 запроса с использованием соединения двух таблиц по равенству;
- 1 запрос с использованием соединения двух таблиц по равенству и условию отбора;
- 1 запрос с использованием соединения по трем таблицам;
- создать копии ранее созданных запросов на соединение по равенству на запросы с использованием внешнего полного соединения таблиц (**JOIN**).
- 1 запрос с использованием левого внешнего соединения;
- 1 запрос на использование правого внешнего соединения;
- 1 запрос с использованием симметричного соединения и удаление избыточности.

Все программные инструкции команд SQL сохранять в файлах с расширением ***.sql** в папке **ФИО_студента/Лаб6**.

Для каждого запроса сформулировать текстовое задание, которое должно быть выполнено к базе данных.

Создать текстовый отчет, в котором отобразить sql-команды разработанных запросов и скриншоты результатов работы из СУБД **SQL Server Management Studio**.

ПРАКТИЧЕСКАЯ РАБОТА №7. СОЗДАНИЕ ЗАПРОСОВ НА ГРУППИРОВКУ И СОРТИРОВКУ ДАННЫХ. ЗАПРОСЫ НА ИЗМЕНЕНИЕ. ИСПОЛЬЗОВАНИЕ ВСТРОЕННЫХ ФУНКЦИЙ.

7.1. Цель работы

Изучить используемый в реляционных СУБД оператор извлечения данных из таблиц SELECT и выполнение группировки и сортировки данных. Изучить синтаксис языка модификации данных. Научится использовать встроенные функции в запросах.

7.2. Исходные данные

Исходными данными является индивидуальное задание и результат предыдущих практических работ.

7.3. Используемые программы

Программа SQL Server Managment Studio.

7.4. Задание

Практическую работу следует выполнять в следующем порядке:

1. Изучить синтаксис создания запросов с использованием функций, группировки и сортировки данных, язык манипулирования данными на примерах запросов, использовать встроенные функции к учебной базе данных **"University.mdf"**.

2. Выполнить в окне **"SQL Editor"** 41 запросов к базе данных **"University.mdf"**, согласно приведенным в практической работе образцам выполнения запросов и сохранить их в файле **"Lab7.sql"** в своей рабочей папке.

Тема 1. Создание запросов с использованием функций

Функции SQL подобны любым другим операторам языка в том смысле, что они производят действия с данными и возвращают результат в качестве своего значения. Функции имеют тип, который определяется типом возвращаемого значения, поэтому можно говорить о числовых, строковых, временных функциях и т. д. От обычных операторов функции отличаются форматом представления:

имя_функции([аргумент[, аргумент]...])

Этот формат допускает, что функции могут иметь ноль, один или более аргументов, причем при отсутствии аргументов круглые скобки не используются.

Имеется два основных класса функций SQL: встроенные и определяемые пользователем.

Встроенными являются функции, предопределенные в SQL. Ко второму классу относятся функции, которые пишутся пользователями на специальном языке, обеспечивающем использование всех возможностей SQL. Каждая СУБД использует для этого свой собственный язык.

SQL Server содержит множество встроенных функций, а также поддерживает создание определяемых пользователем функций.

В SQL определено множество встроенных функций различных категорий. На этом уроке мы рассмотрим:

–агрегатные (или групповые) функции, оперирующие значениями столбцов множества строк и возвращающие одно значение;

–функции одной строки, использующие в качестве аргументов значения столбцов одной строки и возвращающие одно значение.

Встроенные функции (Transact-SQL)

SQL Server содержит множество встроенных функций, а также поддерживает создание определяемых пользователем функций. Категории встроенных функций перечислены на этой странице.

Типы функций

Функция	Описание
Функции, возвращающие наборы строк.	Возвращают объект, который можно использовать так же, как табличные ссылки в SQL-инструкции.
Агрегатные функции	Обрабатывают коллекцию значений и возвращают одно результирующее значение.
Ранжирующие функции	Возвращают ранжирующее значение для каждой строки в секции.
Скалярная функция (описывается далее)	Обрабатывают и возвращают одиночное значение. Скалярные функции можно применять везде, где выражение допустимо.

Скалярные функции

Категория функции	Описание
Функции конфигурации	Возвращают сведения о текущей конфигурации.
Функции преобразования	Поддержка приведения и преобразования типов данных.
Функции работы с курсорами	Возвращают сведения о курсорах.
Функции и типы данных даты и времени	Выполняют операции над исходными значениями даты и времени, возвращают строковые и числовые значения, а также значения даты и времени.
Логические функции	Выполнение логических операций.
Математические функции	Выполняют вычисления, основанные на числовых значениях, переданных функции в виде аргументов, и возвращают числовые значения.
Функции метаданных	Возвращают сведения о базах данных и объектах баз данных.
Функции безопасности	Возвращают данные о пользователях и ролях.
Строковые функции	Выполняют операции со строковым (char или varchar) исходным значением и возвращают строковое или числовое значение.
Системные функции	Выполняют операции над значениями, объектами и параметрами экземпляра SQL Server и возвращают сведения о них.
Системные статистические функции	Возвращают статистические сведения о системе.
Функции обработки текста и изображений	Выполняют операции над текстовыми или графическими исходными значениями или столбцами и возвращают сведения о

Агрегатные функции

Аргументами агрегатных функций могут быть как столбцы таблиц, так и результаты выражений над ними. Агрегатные функции и сами могут включаться в другие арифметические выражения. В стандарте SQL определены следующие виды

агрегатных функций: унарные, бинарные, инверсного распределения, гипотетические функции множеств.

Мы будем рассматривать только определенные в стандарте SQL унарные агрегатные функции. Их перечень представлен в табл. 1.1. Конкретные СУБД расширяют этот список.

AVG - среднее

MIN - минимум

CHECKSUM_AGG - Возвращает контрольную сумму значений в группе. Значения NULL не учитываются.

SUM - сумма

COUNT - количество

STDEV – среднее квадратическое отклонение

COUNT_BIG - Возвращает количество элементов в группе.

STDEVP - Возвращает статистическое стандартное отклонение всех значений в указанном выражении.

GROUPING - Указывает, является ли указанное выражение столбца в списке GROUP BY статистическим или нет. В результирующем наборе функция GROUPING возвращает 1 (статистическое выражение) или ноль (нестатистическое выражение).

VAR - дисперсия

GROUPING_ID - Представляет собой функцию, которая вычисляет уровень группирования.

VARP - Возвращает статистическую дисперсию для заполнения всех значений в указанном выражении.

MAX - максимум

Общий формат унарной агрегатной функции следующий:

имя_функции([ALL | DISTINCT] выражение) [FILTER (WHERE условие)]

где **DISTINCT** указывает, что функция должна рассматривать только различные значения аргумента, а **ALL** — все значения, включая повторяющиеся (этот вариант используется по умолчанию). Фраза **FILTER** позволяет дополнительно отобрать строки таблицы, столбец которой используется в качестве аргумента функции.

Агрегатные функции применяются во фразах **SELECT** и **HAVING**. Здесь мы рассмотрим их использование во фразе **SELECT**. В этом случае выражение в аргументе функции применяется ко всем строкам входной таблицы фразы **SELECT**. Кроме того, во фразе **SELECT** нельзя использовать и агрегатные функции, и столбцы таблицы (или выражения с ними) при отсутствии фразы **GROUP BY**, которую мы рассмотрим в теме 2.

Функция COUNT

Функция **COUNT** имеет два формата. В первом случае возвращается количество строк входной таблицы, во втором случае — количество значений аргумента во входной таблице:

COUNT(*)

COUNT([DISTINCT | ALL] выражение)

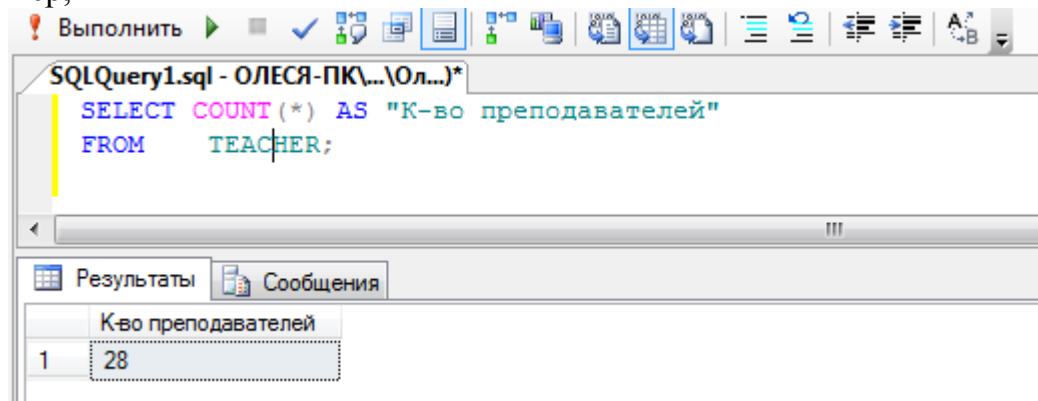
Простейший способ использования этой функции - подсчет количества строк в таблице (всех или удовлетворяющих указанному условию). Для этого используется первый вариант синтаксиса.

Создайте новый запрос, введите sql-запрос, выполните его, сохраните его в рабочую папку ЛАБ7_SQL под именем 1.sql.

Запрос 1. Информация о скольких преподавателях имеется в базе данных?

```
SELECT COUNT(*) AS "К-во преподавателей"  
FROM TEACHER;
```

Чтобы выполнить sql-команду нажмите на панели редактора кнопку Выполнить. В результате выполнения данного кода будет подсчитано кол-во всех преподавателей. Например,



Самостоятельно создать запрос 2. Сколько ассистентов не имеют телефонов?

Запросы с агрегатными функциями можно строить и с использованием нескольких таблиц, так как входная таблица и в этом случае будет только одна.

Самостоятельно создать запрос 3. Сколько кафедр на факультете математики и информатики?

Во втором варианте синтаксиса функции COUNT в качестве аргумента может быть использовано имя отдельного столбца. В этом случае подсчитывается количество либо всех значений в этом столбце входной таблицы, либо только неповторяющихся (при использовании ключевого слова DISTINCT).

Запрос 4. На скольких различных должностях работают преподаватели кафедры «Компьютерные системы и сети»?

```
SELECT COUNT(DISTINCT DOLGNOST)  
FROM KAFEDRA d, TEACHER t  
WHERE d.KOD_KAFEDRU = t.KOD_KAFEDRU AND  
LOWER(d.NAME_KAFEDRU) = 'Компьютерные системы и сети';
```

Функция SUM

Эта агрегатная функция подсчитывает сумму значений аргумента для всех строк входной таблицы. Аргумент должен иметь числовой тип или быть временным промежутком. В качестве аргумента может выступать имя столбца или выражение над столбцами входной таблицы. В этой функции также допускается использовать ключевые слова DISTINCT и ALL. Приведем примеры.

Запрос 5. Какая суммарная ставка всех ассистентов?

```
SELECT SUM(Salary)  
FROM TEACHER  
WHERE LOWER(DOLGNOST) = 'ассистент';
```

Функция AVG

Агрегатная функция AVG подсчитывает среднее значение аргумента для всех строк входной таблицы. Аргумент должен иметь числовой тип или быть временным промежутком. В качестве аргумента может выступать имя столбца или выражение над столбцами входной таблицы. Допускается использовать ключевые слова DISTINCT и ALL. Приведем ряд примеров.

Самостоятельно создать запрос 6. Какая средняя ставка среди всех преподавателей?

Самостоятельно создать запрос 7. Какое среднее значение ставки в вузе?

В данном запросе используйте ключевое слово DISTINCT, чтобы применить AVG не ко всем имеющимся в таблице TEACHER ставкам, а только к различным значениям ставки.

Функции MIN и MAX

Эти функции позволяют находить максимальное (MAX) и минимальное (MIN) значения аргумента для всех строк входной таблицы. Хотя и в этом допускается использование ключевых слов DISTINCT и ALL, они не оказывают влияния на результат. Аргумент этих функций может быть любого типа, для которого определено упорядочение, то есть числовой, строковый и временной.

Запрос 8. Какова максимальная зарплата преподавателя ?

```
SELECT MAX(Salary + Rise)  
FROM TEACHER;
```

Самостоятельно создать запрос 9. Когда в последний раз (максимальная дата приема на работу) принимали на работу преподавателя на кафедре информатики?

Выражения с использованием агрегатных функций

Агрегатные функции не только могут иметь выражение в своем аргументе, но и сами могут использоваться в выражениях.

Запрос 10. Вывести процентное соотношение суммарной ставки к суммарной зарплате и наоборот.

```
SELECT SUM(Salary)*100/SUM(Rise) AS "Процент зарплаты к зарплате",  
SUM(Rise)*100/SUM(Salary) AS "Процент зарплаты к ставке"  
FROM TEACHER;
```

Однострочные функции

Напомним, что эти функции используют в качестве аргумента одно значение (одного столбца одной строки таблицы) и возвращают в качестве своего результата также единственное значение. Мы рассмотрим эти функции по типам их аргументов.

Строковые функции

Эти функции используют в качестве аргумента строку символов и в качестве результата возвращают также символьную строку. Стандарт SQL предлагает варианты таких функций и для двоичных строк.

Функции UPPER, LOWER

Эти функции мы уже рассматривали и многократно использовали. Они имеют следующий формат:

UPPER(строка)

LOWER(строка)

Приведем для них примеры .

Запрос 11. Вывести фамилии всех преподавателей прописными буквами.

```
SELECT UPPER(NAME_TEACHER) AS "Все прописные"  
FROM TEACHER;
```

Аналогично можно вывести все фамилии преподавателей строчными буквами.

Числовые функции над числами

Эти функции возвращают числовые значения на основании заданных в аргументе значений того же типа. Числовые функции используются для обработки данных, а также в условиях их поиска. Стандарт SQL предлагает ряд числовых функций с очевидной семантикой. Часть функций перечислены ниже:

ABS абсолютное значение

DEGREES Возвращает для значения угла в радианах соответствующее значение в градусах.

RAND – Возвращает псевдослучайное значение типа float от 0 до 1.

EXP экспонента

ROUND - Возвращает числовое значение, округленное до указанной длины или точности.

FLOOR Возвращает наибольшее целое число, меньшее или равное указанному числовому выражению.

LOG логорифм

SIN - синус

LOG10 десятичный логорифм

SQRT – корень квадратный

PI число 3.14

SQUARE – квадрат числа

POWER Возвращает значение указанного выражения, возведенное в заданную степень.

TAN - тангенс

Особой функцией является WIDTH_BUCKET, с помощью которой можно легко строить гистограммы:

WIDTH_BUCKET(число, минимум, максимум, количество)

Некоторые СУБД расширяют приведенный выше набор функций, включая другие числовые функции, например вычисления обычных и гиперболических тригонометрических функций.

Временные функции

Эти функции используют в качестве аргумента типы даты, времени, временной отметки или временного промежутка. Тип возвращаемого значения не всегда соответствует типу аргумента.

Функции даты и времени в Transact-SQL делятся на

Функции, получающие значения системной даты и времени

Функции, получающие компоненты даты и времени

Функции, получающие значения даты и времени из их компонентов

Функции, получающие разность даты и времени

Функции, изменяющие значения даты и времени

Функции, устанавливающие или получающие формат сеанса
Функции, проверяющие значения даты и времени.

Функции, получающие компоненты даты и времени.

Функция извлекает из операнда указанный компонент и возвращает его в виде числа.

DATENAME (datepart , date)

Здесь date - это выражение временного типа, а datepart - временная единица, которая может иметь одно из следующих значений: YEAR, MONTH, DAY, HOUR, MINUTE, SECOND и т.д.

DATEPART (datepart,date) - Возвращает целое число, представляющее указанный компонент datepart указанной даты date.

DAY (date) - Возвращает целое число, представляющее день указанной даты date.

MONTH (date) - Возвращает целое число, представляющее месяц указанной даты date.

YEAR (date) - Возвращает целое число, представляющее год указанной даты date.

Рассмотрим пример.

Запрос 12. Вывести фамилии всех преподавателей родившихся в 1979 году.

```
SELECT Name_teacher, BIRTHDAY  
FROM TEACHER  
WHERE DATENAME(YEAR, BIRTHDAY)=1979;
```

Функции, получающие значения системной даты и времени

Функция CURRENT_TIMESTAMP - Возвращает значение типа datetime2(7), которое содержит дату и время компьютера, на котором запущен экземпляр SQL Server. Смещение часового пояса не включается.

Эта функция возвращает текущую дату. Аргументов она не имеет.

Функция GETDATE () Возвращает значение типа datetime2(7), которое содержит дату и время компьютера, на котором запущен экземпляр SQL Server. Смещение часового пояса не включается.

Функция GETUTCDATE () Возвращает значение типа datetime2(7), которое содержит дату и время компьютера, на котором запущен экземпляр SQL Server. Возвращаемые дата и время отображаются в формате UTC.

Многие СУБД существенно расширяют список функций, оперирующих датой и временем. Далее мы приведем некоторые из важных функций этого типа, которые используются в Oracle. Напоминаем, что тип DATE в Oracle содержит в себе как дату, так и время.

Функции, получающие значения даты и времени из их компонентов

Функция DATEADD (datepart, number , date) Возвращает новое значение datetime, добавляя интервал к указанной части datepart заданной даты date.

Добавляет к дате, указанной в первом аргументе, количество месяцев второго аргумента.

Dateadd (компонент, кол-во , дата)

Здесь кол-во - это количество прибавляемых лет, месяцев, дней и т.д., а компонент - временная единица, которая может иметь одно из следующих значений: YEAR, MONTH, DAY, HOUR, MINUTE, SECOND.

Например, DATEADD(month, 1, '2006-08-30')

Запрос 13. Осуществить пересчет даты приема на работу преподавателя на фамилию начинающуюся на букву С в сторону увеличения на 3 месяца.

```
SELECT NAME_TEACHER, DATA_HIRE AS ' Дата приема ',  
DATEADD(month, 3, DATA_HIRE) AS ' Плюс 3 месяца '  
FROM TEACHER  
WHERE (NAME_TEACHER) LIKE 'С%';
```

Функция EOMONTH

EOMONTH (start_date [, month_to_add])

Возвращает дату последнего дня того месяца, который указан в аргументе. Обычно используется для определения, сколько дней осталось до конца месяца.

LAST_DAY(дата)

Функция DATEDIFF

DATEDIFF (datepart , startdate , enddate)

Возвращает количество пересеченных границ (целое число со знаком), указанных аргументом datepart, за период времени, указанный аргументами startdate и enddate.

Запрос 14. Например, если вы хотите узнать, сколько месяцев уже проработал Статывка, можно выполнить такой запрос:

```
SELECT 'Статывка проработал ' ||  
ROUND(DATEDIFF(month, GETDATE(), DATA_HIRE), 1) ||  
' месяцев' AS "Стаж Статывки"  
FROM TEACHER  
WHERE NAME_TEACHER LIKE 'Статыв%';
```

Функция NEXT_DAY

Возвращает ближайшую к первому параметру дату, в которой название дня недели совпадает с указанным во втором параметре.

NEXT_DAY(дата, день_недели)

Функции преобразования

Стандарт SQL предлагает единственную функцию преобразования данных из одного типа в другой — это функция CAST.

Функция CAST и CONVERT

Производит преобразование выражения, заданного первым аргументом, в тип, заданный вторым аргументом. Преобразование допускается только для определенных пар типов данных.

CAST (expression AS data_type [(length)])

CONVERT (data_type [(length)] , expression [, style])

Например

CAST(10.3496847 AS money)

CAST(10.6496 AS int)

Тема 2. Группировка и сортировка

В этом уроке мы рассмотрим еще три фразы предложения SELECT, а именно: **HAVING, GROUP BY и ORDER BY.**

Первая из них позволяет группировать строки таблицы и применять к созданным группам агрегатные функции.

Рассмотрим простейшие варианты группировки. Фраза HAVING используется вместе с фразой GROUP BY и позволяет формулировать условия на группах строк для дополнительного отбора.

Наконец, фраза ORDER BY позволяет сортировать строки результирующей таблицы.

Запросы с группировкой строк

Часто при создании отчетов появляется необходимость в формировании промежуточных итоговых значений, то есть относящихся к данным не всей таблицы, а ее частей.

Именно для этого предназначена фраза GROUP BY. Она позволяет все множество строк таблицы разделить на группы по признаку равенства значений одного или нескольких столбцов (и выражений над ними).

Фраза GROUP BY должна располагаться вслед за фразой WHERE (если она отсутствует, то за фразой FROM).

Общий синтаксис фразы GROUP BY следующий:

GROUP BY выражение[, выражение]...

При наличии фразы GROUP BY фраза SELECT применяется к каждой группе, сформированной фразой группировки. В этом случае и действие агрегатных функций, указанных во фразе SELECT, будет распространяться не на всю результирующую таблицу, а только на строки в пределах каждой группы. Каждое выражение в списке фразы SELECT должно принимать единственное значение для группы, то есть оно может быть:

- константой;
- агрегатной функцией, которая оперирует всеми значениями аргумента в пределах группы и агрегирует их в одно значение (например, в сумму);
- выражением, идентичным стоящему во фразе GROUP BY;
- выражением, объединяющим приведенные выше варианты.

Рассмотрим возможности фразы GROUP BY, переходя от простых вариантов ее использования к более сложным.

Группировка по одному столбцу

Группировка по значениям одного столбца является самым простым вариантом использования фразы GROUP BY. Приведем примеры.

Запрос 15. Для каждого корпуса подсчитать количество находящихся в нем кафедр.

```
SELECT NUM_KORPUSA AS "Корпус",  
COUNT(*) AS "К-во кафедр"  
FROM KAFEDRA  
GROUP BY NUM_KORPUSA ;
```

Самостоятельно создать запрос 16. Для каждой из должностей указать суммарный фонд заработной платы.

Если в запросе используются фразы и WHERE, и GROUP BY, строки, не удовлетворяющие условию фразы WHERE, исключаются до выполнения группировки. Вследствие этого группировка производится только по тем строкам, которые удовлетворяют условию.

В случае многотабличных запросов сначала производится соединение таблиц, а затем их группировка. Приведем примеры.

Самостоятельно создать запрос 17. Для каждого факультета, расположенного в корпусе 1, вывести количество групп и общее количество студентов по каждой кафедре.

Группировка по нескольким столбцам

SQL позволяет группировать строки таблицы и по нескольким столбцам. В этом случае имена столбцов перечисляются во фразе GROUP BY через запятую.

Запрос 18. Для каждого факультета, расположенного в корпусе 1, вывести сколько учится студентов по каждой группе.

```
SELECT f.Name_faculteta,  
s."GROUP", count(s."GROUP") AS "Кол-во студентов в группе"  
FROM FACULTET f, KAFEDRA d, STUDENT s  
WHERE f.KOD_FACULTETA = d.KOD_FACULTETA AND  
d.KOD_kafedru = s.KOD_kafedru AND  
d.NUM_KORPUSA = '1'  
GROUP BY f.Name_faculteta,s."GROUP";
```

Самостоятельно создать запрос 19. Для каждой кафедры и должности вывести суммарную и среднюю зарплату преподавателей.

Даже при группировке по двум и более столбцам этот вариант фразы GROUP BY обеспечивает только один уровень группировки. Так, приведенный выше запрос обеспечивает только одну итоговую строку для пары значений кафедра-должность.

Использование выражений

Хотя стандарт SQL не допускает группировку по выражениям над столбцами, некоторые СУБД такую возможность предоставляют. В этом случае во фразе SELECT также можно использовать выражение группировки, однако нельзя выводить по отдельности столбцы, участвующие в этом выражении.

Запрос 20. Для каждого значения зарплаты, не превышающего 1500, вывести это значение и количество преподавателей, такую зарплату получающих.

```
SELECT Salary + Rise, COUNT(*)  
FROM TEACHER  
WHERE Salary + Rise <= 1500  
GROUP BY Salary + Rise;
```

Вложение агрегатных функций

Если фраза GROUP BY в запросе отсутствует, то во фразе SELECT нельзя вкладывать агрегатные функции друг в друга. Например, следующий запрос приведет к ошибке:

```
SELECT AVG(MIN(Salary))  
FROM TEACHER;
```

Однако при наличии фразы GROUP BY такое вложение допускается. Оно интерпретируется следующим образом: сначала для каждой группы выполняется вложенная агрегатная функция, затем к полученной таким образом промежуточной таблице применяется внешняя агрегатная функция. Двойное вложение, например MAX(AVG(MIN(Salary))), недопустимо. Приведем пример.

Запрос 21. Вывести среднее значение среди минимальных и максимальных ставок для каждой группы преподавателей, занимающих одну должность, а также минимальное и максимальное значения среди средних ставок.

```
SELECT AVG(MIN(Salary)) AS AVG_MIN,
       AVG(MAX(Salary)) AS AVG_MAX,
       MIN(AVG(Salary)) AS MIN_AVG,
       MAX(AVG(Salary)) AS MAX_AVG
FROM   TEACHER
GROUP BY Dolgnost ;
```

Условие отбора групп

Предположим, что нужно вывести номера кафедр, у которых суммарное количество работающих профессоров более 1. Приведенная ниже формулировка запроса является неверной:

```
SELECT KOD_kafedru
FROM   TEACHER
WHERE  count(dolgnost) > 1 and dolgnost='профессор'
GROUP BY KOD_kafedru;
-----
WHERE  count(dolgnost) > 3 and dolgnost='профессор';
*
```

Ошибка в строке 3;

CRA-00934: групповая функция здесь не разрешена

Дело в том, что фраза WHERE проверяет на соответствие условию строки исходных таблиц, а мы указали в ней агрегатную функцию. Для отбора строк среди полученных групп следует применять фразу HAVING. Она играет такую же роль для групп, что и фраза WHERE для исходных таблиц, и может использоваться лишь при наличии фразы GROUP BY. В предложении SELECT фразы WHERE, GROUP BY и HAVING обрабатываются в следующем порядке.

Фразой WHERE отбираются строки, удовлетворяющие указанному в ней условию.

Фраза GROUP BY группирует отобранные строки.

Фразой HAVING отбираются группы, удовлетворяющие указанному в ней условию. В связи с вышесказанным, предыдущий запрос необходимо записать так.

Перепишем тогда запрос так:

Запрос 22. Вывести номера кафедр, у которых суммарное количество работающих профессоров более 1.

```
SELECT KOD_kafedru as "Номер кафедры" ,Count(*) as "Кол-во профессоров
на кафедре"
FROM   TEACHER
WHERE  dolgnost='профессор'
GROUP BY KOD_kafedru
having count(dolgnost) > 1 ;
```

Использование столбцов группировки во фразе HAVING

Рассмотрим использование во фразе HAVING условий отбора, заданных для группируемых столбцов (или выражений над ними). Для этого усложним предыдущий запрос.

Запрос 23. Вывести названия кафедр факультета математики и информатики, на которых работают один и более профессоров. Указать также количество профессоров и их суммарную зарплату.

```
SELECT d.Name_kafedru, Count(*), SUM(t.salary + t.Rise)
FROM FACULTET f, KAFEDRA d, TEACHER t
WHERE f.KOD_FACULTETA = d.KOD_FACULTETA AND
d.KOD_kafedru = t.KOD_kafedru AND
LOWER(f.Name_faculteta) = 'математики и информатики' AND
LOWER(t.Dolgnost) = 'профессор'
GROUP BY d.Name_kafedru
HAVING COUNT(*) > 0;
```

Фраза HAVING без фразы GROUP BY

Выше мы указали, что фраза HAVING может использоваться лишь при наличии фразы GROUP BY. Из этого правила синтаксис SQL допускает только одно исключение: когда вся таблица интерпретируется как одна группа. В этом случае в списке фразы SELECT можно использовать только константы, агрегатные функции и выражения над ними. Приведем примеры.

Запрос 24. Если суммарная зарплата всех преподавателей превышает 15 000, вывести их минимальную ставку, максимальную надбавку и суммарную зарплату.

```
SELECT MIN(Salary), MAX(Rise), SUM(Salary + Rise)
FROM TEACHER
HAVING SUM(Salary + Rise) > 15000;
```

При наличии фразы WHERE сначала производится отбор строк согласно ее условию, и только после этого применяется условие фразы HAVING.

Запрос 25. Если суммарная зарплата всех ассистентов превышает 2500, вывести их среднюю ставку, среднюю надбавку и суммарную зарплату.

```
SELECT AVG(Salary), AVG(Rise), SUM(Salary + Rise)
FROM TEACHER
WHERE LOWER(Dolgnost) = 'ассистент'
HAVING SUM(Salary + Rise) > 2500;
```

На практике фраза HAVING очень редко используется без фразы GROUP BY, из-за чего такая возможность предоставляется не во всех СУБД.

Сортировка результирующих строк

Как мы уже отмечали, строки в таблицах базы данных неупорядочены. Также неупорядочены и строки результирующей таблицы запроса, однако для их упорядочения в предложении SELECT можно воспользоваться фразой ORDER BY. Она сортирует по значению указанных в ней столбцов (и выражений над столбцами) строки результирующей таблицы запроса. Синтаксис этой фразы следующий:

ORDER BY спецификация_сортировки[. спецификация_сортировки]...
где спецификация_сортировки имеет такой синтаксис:

выражение_сортировки [направление_сортировки] [положение_NULL]

Сортировать можно по столбцам (выражениям) тех типов, для которых определены операции сравнения. Это относится, в частности, к символьным строкам, числам и временным значениям. Можно указывать направление сортировки и место расположения строк, имеющих значение NULL для выражений сортировки.

Далее в этом уроке мы рассмотрим общие способы упорядочения результирующих строк.

Сортировка по столбцу или выражению

Сортировать строки результирующей таблицы запроса можно по отдельным столбцам, совокупности столбцов, а также по одному или нескольким выражениям над столбцами. Ниже рассматриваются все эти варианты.

Сортировка по столбцу

Простейший вариант сортировки - это сортировка по одному из столбцов результирующей таблицы.

Запрос 26. Вывести алфавитный список фамилий профессоров и доцентов.

```
SELECT NAME_TEACHER
FROM TEACHER
where LOWeR(Dolgnost ) ='профессор' OR
LOWeR(Dolgnost ) = 'доцент'
ORDER BY NAME_TEACHER;
```

Сортировка по выражению над столбцами

Упорядочивать строки можно не только по значению столбца, но и по значению выражения над столбцами.

Запрос 27. Вывести фамилии ассистентов и их зарплату по ее возрастанию.

```
SELECT Name_teacher, Salary + Rise
FROM TEACHER
WHERE LOWeR(Dolgnost ) = 'ассистент'
ORDER BY Salary + Rise;
```

Направление сортировки

Во всех до сих пор приводимых примерах сортировка производилась в порядке возрастания значений. В SQL такой порядок определен по умолчанию. Однако есть возможность и явно указать направление сортировки с помощью ключевых слов ASC (по возрастанию) и DESC (по убыванию), которые следует располагать после имени сортируемого столбца (выражается).

Запрос 28. Вывести фамилии ассистентов и дату их приема на работу по возрастанию даты.

```
SELECT Name_teacher, Data_hire
FROM TEACHER
WHERE LOWeR(Dolgnost) = 'ассистент'
ORDER BY Data_hire ASC;
```

Самостоятельно создать запрос 29. Вывести фамилии доцентов в обратном алфавитном порядке и их зарплату.

Тема 3. Внесение изменений в базу данных

7.4.1. Добавление новых данных

Новые данные добавляются оператором INSERT. Наименьшей единицей информации, которую можно добавить в реляционную базу данных, является одна строка таблицы.

Немного упрощенный синтаксис оператора INSERT имеет вид:

```
INSERT INTO Имя_Таблицы [(Колонка [, Колонка ...])]
{VALUES(<величина> [, <величина> ...]) | <оператор SELECT>;}
<величина> = { :Переменная | <константа> | <выражение>
| <функция> | udf([<величина> [, <величина> ...]])
| NULL | USER }
```

<константа> = Число | 'Строка'

<функция> = CAST(<величина> AS <тип данных>)
UPPER(<величина>)
GEN_ID(Имя_Генератора, <величина>)

<выражение> = SQL выражение, возвращающее единичное значение

В этом описании можно выделить два варианта оператора:

1. Вставка одной строки. Для этого после ключевого слова VALUES в круглых скобках указывают вставляемые величины.
2. Вставка в таблицу нескольких строк, выбранных с помощью оператора SELECT *.

В этой практической работе рассматривается только первый вариант оператора INSERT.

Пример, когда в качестве вставляемых величин применены константы:

```
INSERT INTO Person(Pr_ID, Pr_LastName, Pr_FirstName)
VALUES(150, 'Иванов', 'Петр');
```

7.4.2. Удаление существующих данных

Для удаления строк из таблицы используется оператор DELETE. Вот его упрощенный синтаксис:

```
DELETE FROM Имя_Таблицы
[WHERE <условие поиска>;]
```

<условие поиска> = как в операторе SELECT

Если не использовать предложение WHERE, то будут удалены все строки в таблице.

-- Удаление всех служащих:

```
DELETE FROM Employee;
```

-- Удаление всех людей с номерами 150 и больше:

```
DELETE FROM Person WHERE Pr_ID >= 150;
```


Отбирать строки для удаления не обязательно только на основании содержимого этих строк. Можно составить условие для удаляемых строк, опираясь на данные из других таблиц. Для составления таких условий необходимо сначала изучить оператор SELECT.

7.4.3. Обновление существующих данных

Оператор UPDATE обновляет значения одного или нескольких столбцов в выбранных строках одной таблицы. Строки для обновления указываются в предложении WHERE. Если пропустить предложение WHERE, то изменятся все строки таблицы.

```
UPDATE Имя_Таблицы
SET Колонка      =      <величина>[,
Колонка      =      <величина>...]
[WHERE <условие поиска>]
<величина> = { Колонка | :Переменная | <константа>
| <выражение> | <функция>
| udf([<величина> [, <величина> ...]]) | NULL | USER}
<выражение> = SQL выражение, возвращающее единичное значение
<условие поиска> = как в операторе SELECT
```

Примеры:

-- Увеличить зарплату всем служащим на 10%:

```
UPDATE Employee
SET Salary = 1.1*Salary;
```

/* Увеличить зарплату всем служащим, которые имеют зарплату меньше 10000 на 15%: */

```
UPDATE Employee
SET Salary = 1.15*Salary;
WHERE Salary <= 10000;
```

Отбирать строки для изменения, как и для удаления, можно с использованием подчиненного запроса SELECT, который позволит учитывать в условии поиска изменяемых строк данные из других таблиц.

Например, можно выполнить такой запрос: увеличить зарплату на 10% всем служащим, работающим в отделе продаж, которые обслужили за последний месяц клиентов больше чем в полтора раза, чем в среднем по их отделу.

7.5. Задание

Добавление новых строк

Предложение INSERT вставляет строки в таблицу базы данных. Есть три разновидности этой команды:

```
INSERT VALUES
INSERT SELECT
INSERT DESALT VALUES
```

Первая из них производит вставку в таблицу явно заданной строки, вторая разновидность – вставку группы строк, выбранных в результате выполнения запроса, а третья — вставку значений по умолчанию.

Вставка отдельных строк

Предложение INSERT... VALUES выполняет вставку в таблицу одной строки. Его удобно использовать для небольших операций, когда в таблицу нужно вставить несколько строк. Синтаксис этого предложения следующий:

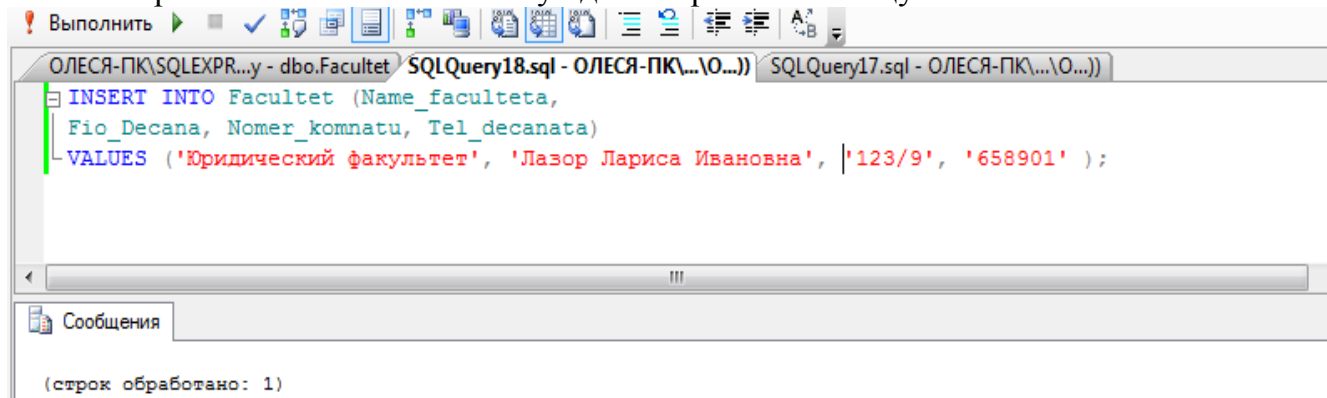
```
INSERT INTO имя_таблицы [<имя_столбца[, имя_столбца]...]  
VALUES (значение[, значение]...);
```

Указание вставляемых столбцов

Этот формат предполагает указания имени таблицы, в которую производится вставка, списка имен столбцов, в которые будут вставляться значения, и списка собственно вставляемых в строку значений. При этом следует придерживаться следующих правил:

- вставляемые данные должны согласовываться с типами данных указанных столбцов;
- размеры данных должны соответствовать размерам столбцов;
- порядок данных во фразе VALUES должен соответствовать порядку перечисления столбцов.

Запрос 30. Выполнить вставку одной строки в таблицу FACULTET



В таблицу вставляется строка со значениями, указанными в списке фразы VALUES, причем расположение значений в списке соответствует расположению соответствующих столбцов в списке столбцов таблицы.

Примечание. Чтобы данные были добавлены, не забудьте нажать на кнопки , а затем для подтверждения внесенных изменений в таблицу. Просмотрите внесенные изменения в таблицу FACULTET.

В этом примере мы перечислили столбцы в том порядке, в каком они были определены при создании таблицы, однако это не обязательно. При желании порядок перечисления имен столбцов в команде INSERT можно изменить.

Список имен столбцов может быть не полным. Можно указывать только те из них, значения которых известны для вставляемой строки, Столбцы, отсутствующие в списке, будут принимать значения NULL для вводимой строки.

Самостоятельно создайте запрос 31. Выполнить вставку одной строки в таблицу KAFEDRA для столбцов name_kafedru, fio_zavkaf, kod_faculteta с данными 'Психологии', 'Иванова', 5.

В этом примере в вводимой строке отсутствуют значения столбцов Nomer_komnatu, Num_korpusa и Tel_kafedru. В базе данных они примут значение NULL.

Поддержка ограничений целостности

Помните, некоторые из столбцов или наборов столбцов могут иметь ограничения целостности PRIMARY KEY и NOT NULL. Такие столбцы не могут принимать значения NULL.

Приведенные выше рассуждения относятся ко всем ограничениям целостности, определенным для таблиц. При попытке ввода данных (как, впрочем, и при обновлении и удалении) СУБД проверяет возможное нарушение объявленных ограничений целостности. И если это так, команда будет отклонена с выдачей соответствующего уведомления.

Использование выражений

В качестве вставляемых значений могут использоваться выражения.

Самостоятельно создайте запрос 32. Ввести в таблицу TEACHER данные (50, 10, 'Капуста Леонид Владимирович', 1271, 1271/3, 'доцент', GETDATE()-1)

Здесь мы указали, что надбавка равна третьей части ставки (1271 / 3), а дата приема на работу на один день меньше текущей даты (CURRENT_DATE -1).

Результат запроса в качестве вставляемого значения

Вместо вставляемого значения можно использовать запрос. Это оказывается очень удобным в том случае, когда вставляемое значение присутствует в базе данных.

Запрос 33. Например, в следующем предложении в качестве фамилии заведующего вновь вставляемой кафедры выбирается фамилия декана факультета «Компьютерных наук и технологий».

```
INSERT INTO KAFEDRA (name_kafedru, kod_faculteta, fio_zavkaf)  
VALUES ( 'Философии', 5, (SELECT fio_decana FROM FACULTET  
WHERE LOWER(Name_faculteta) = 'международный'));
```

Обновление существующих данных

Целью предложения обновления является изменение значений отдельных столбцов всех или удовлетворяющих указанному условию строк таблицы. Упрощенный синтаксис предложения следующий:

```
UPDATE имя_таблицы [[AS] синоним]  
SET имя_столбца = выражение[, имя_столбца = выражение]...  
[WHERE условие];
```

Его элементы означают следующее:

имя_таблицы — имя обновляемой таблицы;

синоним — синоним обновляемой таблицы для ссылки на нее в подзапросе;

имя_столбца - имя обновляемого столбца;

выражение - допустимое в SQL выражение соответствующего типа, значение которого присваивается обновляемому столбцу;

условие - допустимое в SQL выражение условия, которое используется для отбора обновляемых строк.

По одному предложению UPDATE обновлению подвергаются строки только одной базовой таблицы.

Обновление всех строк

Как видно из определения синтаксиса команды UPDATE, фраза WHERE является факультативной. При ее отсутствии все строки таблицы подвергаются обновлению согласно фразе SET.

Во фразе SET можно одновременно изменять значения нескольких столбцов таблицы.

Запрос 34. Например, в следующем примере всем преподавателям увеличивается ставка на 12 % и надбавка на 7 %:

UPDATE TEACHER

SET Salary = Salary + Salary * 0.12, Rise = Rise + Rise * 0.08;

Во фразе SET в правой части оператора присваивания может использоваться любое допустимое в SQL выражение того же типа, что и столбец, имя которого приведено слева от оператора присваивания. Если в этом выражении используется имя столбца целевой таблицы, для вычисления выражения применяется значение этого столбца в текущей строке, которое было перед обновлением.

Примечание. Чтобы данные были добавлены, не забудьте нажать на кнопки , а затем  для подтверждения внесенных изменений в таблицу. Просмотрите внесенные изменения в таблицу TEACHER.

Обновление по условию

Данный вариант использует фразу WHERE. В этом случае обновляются столбцы только тех строк таблицы, на которых выполняется условие фразы WHERE. Рассмотрим несколько примеров.

Запрос 35. Увеличить всем ассистентам зарплату и надбавку на 10 %:

UPDATE TEACHER

SET Salary = Salary * 1.1, Rise = Rise * 1.1

WHERE LOWER(Dolgnost) = 'ассистент';

Самостоятельно создать запрос 36. Установить, что деканат юридического факультета переместился в комнату 232 8 корпуса.

Подзапросы во фразе WHERE

Во фразе WHERE можно использовать подзапросы, как мы это делали в предложении SELECT. Это дает возможность отбирать строки для обновления на основе информации из других таблиц.

Запрос 37. Например, увеличить ставку всех преподавателей кафедры прикладной математики факультета математики и информатики в полтора раза:

UPDATE TEACHER

SET Salary = Salary * 1.5

WHERE KOD_kafedru = (SELECT KOD_kafedru FROM KAFEDRA

WHERE LOWER(Name_kafedru) = 'прикладная математика');

Подзапросы во фразе SET

До сих пор новые значения представляли собой константы или выражения с использованием значений обновляемой строки. Однако если такие значения присутствуют в других строках обновляемой таблицы или вообще в других таблицах, можно воспользоваться подзапросом. В этом случае допускается использовать две формы фразы SET:

SET {имя_столбца | (список_имен_столбцов)} = (подзапрос)

В обоих вариантах подзапрос должен возвращать одну строку. В первом случае он также должен возвращать значение одного столбца, а во втором возвращаемая строка должна содержать столько значений, сколько столбцов приведено в списке имен столбцов. При этом производится присвоение значений строки подзапроса соответствующим столбцам из списка слева.

Запрос 38. Установить всем ассистентам надбавку, равную 70 % текущей средней надбавки по вузу.

UPDATE TEACHER

SET Rise = (SELECT SUM(Rise) * 0.7 / COUNT(*) FROM TEACHER)
WHERE LOWER(Dolgnost) = 'ассистент';

Удаление существующих строк

Удалять строки из таблицы можно с помощью предложения DELETE. Оно удаляет только строки целиком, а не индивидуальные значения столбцов. Синтаксис команды следующий:

DELETE FROM имя_таблицы [[AS] синоним]
[WHERE условие];

При использовании предложения DELETE вы прежде всего обнаружите, что предупреждающая подсказка, как правило, не выдается. Обычно, когда пользователь удаляет какой-либо объект операционной среды, он получает сообщение типа «Вы уверены (Д/Н)?». В системах, поддерживающих SQL, строки удаляются без такого сообщения. Поэтому будьте внимательны.

В зависимости от применения фразы WHERE предложение DELETE позволяет удалить отдельную строку, несколько или все строки таблицы. Строки могут быть и не удалены. При использовании предложения DELETE помните о следующем:

нельзя удалить значение отдельного столбца (используйте для этого предложение UPDATE);

как и предложения INSERT и UPDATE, удаление строк может нарушить ограничения целостности;

сама таблица не удаляется (используйте для этого предложение DROP TABLE).

Удаление всех строк таблицы

Чтобы удалить все содержимое таблицы, не нужно использовать фразу WHERE. Помните, что вы удаляете не саму таблицу, а только все ее строки.

Запрос 39. Удалить содержимое таблицы Сотрудники базы данных Educator.

use Educator
DELETE FROM Сотрудники;

Удаление по условию

Обычно нужно удалять только некоторые строки из таблицы. Чтобы определить, какие строки будут удалены, нужно использовать условие во фразе WHERE. Приведем несколько примеров.

Самостоятельно создать запрос 40. Удалить сведения об ассистентах, которые были приняты на работу до 01.01.1986.

Удаление одной строки

Чтобы удалить одну конкретную строку, нужно сформулировать условие таким образом, чтобы оно идентифицировало эту единственную строку. Обычно для этого в условии используются первичный ключ таблицы или уникальный набор столбцов.

Самостоятельно создать запрос 41. Удалить всех преподавателей под фамилией Швец.

Задание для практической работы №7

Для созданной базы данных, согласно номеру варианта, самостоятельно создать **на языке Transact-SQL 14 многотабличных запросов:**

- 1 запрос с использованием функции **COUNT**;
- 1 запрос с использованием функции **SUM**;
- 1 запрос с использованием функций **UPPER, LOWER**;
- 1 запрос с использованием временных функций;
- 1 запрос с использованием группировки по одному столбцу;
- 1 запрос на использование группировки по нескольким столбцам;
- 1 запрос с использованием условия отбора групп **HAVING**;
- 1 запрос с использованием фразы **HAVING** без фразы **GROUP BY**;
- 1 запрос с использованием сортировки по столбцу;
- 1 запрос на добавление новых данных в таблицу;
- 1 запрос на добавление новых данных по результатам запроса в качестве вставляемого значения;
- 1 запрос на обновление существующих данных в таблице;
- 1 запрос на обновление существующих данных по результатам подзапроса во фразе **WHERE**;
- 1 запрос на удаление существующих данных.

Все программные инструкции команд SQL сохранять в файлах с расширением ***.sql** в папке **ФИО_студента/Лаб7**.

Для каждого запроса сформулировать текстовое задание, которое должно быть выполнено к базе данных.

Создать текстовый отчет, в котором отобразить sql-команды разработанных запросов и скриншоты результатов работы из СУБД **SQL Server Management Studio**.